Hardware System Design and Performance Report for PrecisionBeam

Table of Contents:

1. Introduction
2. Client Background and Requirements
3. Current Recommendation Diagnosis
4. Proposed System Specification
    - 4.1 Processor
    - 4.2 Memory
    - 4.3 Storage
    - 4.4 Motherboard, OS, Network, Graphics and I/O
    - 4.5 Chassis, PSU and Cooling
    - 4.6 Reused Components
    - 4.7 System Integration
    - 4.8 Error Detection, Correction and Maintenance
    - 4.9 Summary
5. Performance Impact Discussion
6. Alternative Architecture Considered
7. Program Overview and Purpose
8. Program Performance Analysis
    - 8.1 Methodology
    - 8.2 Testing
    - 8.3 Theories, Benchmarking and Analysis
    - 8.4 Analysis and Results
9. Conclusion
10. Reference
11. Appendix A
    - 14.1 Appendix A.1 – Program Execution Output
    - 14.2 Appendix A.2 – Program Source Code

1. Introduction

Modern particle experiment facilities increasingly depend on high-throughput computing systems to process large volumes of detector data in real time, while single-core performance improvements have reached a diminishing return phase. Therefore, parallel computing has become the field of growing interest, where workloads can be divided across multiple processing cores and specialized hardware to achieve higher throughput.

This report presents the computer system designed for PrecisionBeam, a small fictional particle collider facility, whose data-processing capacity has been overwhelmed following the recent particle detector upgrade. Their 2008 computer system can no longer filter and store all incoming events, as the hit rate rose substantially, wasting expensive beam time and forcing a lowered intensity.

The report showcases the diagnosis of the current bottleneck and specification for a replacement system. Each choice is supported with justification and alternatives, compared against the current one using performance metrics and a benchmark program, utilizing parallelization, to demonstrate performance improvement and support of the new system in the facility's requirements.

2. Client Background and Requirements

PrecisionBeam is a small, university-based collider facility aimed at monitoring known particles and their behavior. Their particle detector produces a continuous stream of data recording events. The events include genuine particle interactions, as well as noise from background events, generated by electronic noise or cosmic radiation (Wielers, 2014). Due to only a portion of the recorded data being relevant, it must be filtered and later sent to an analysis facility.

Recently, an upgrade of the detector has been introduced in the facility, substantially increasing the generation rate of the events. The upgrade also included a new PCIe data acquisition (DAQ) card, custom made to the detector, which was installed in the

existing server's only PCIe slot, allowing communication with the computer. This, while improving research capabilities for the facility, may also be causing loss of potentially useful detector data and reduces the amount of beam time that can be used effectively. The reason behind it is an overwhelmed computer infrastructure, installed in 2008 using low-cost and some older-generation components to fit the budget, which runs a single-core processor to perform filtering and storage operation. Limits like these aren't uncommon, especially in larger facilities, like LHCb, where the readout and trigger system limited the luminosity that could be recorded (LHCb Collaboration, 2015). Furthermore, the facility was forced to use a fraction of intensity of their particle accelerator (reported around 20%) to prevent overwhelming the system.

The facility has been recommended to increase their memory to support the new detector and restore full beam intensity. However, due to their tight, university-funded budget, the facility is obligated to seek an independent assessment before committing to this expenditure. The focus is therefore placed on identifying system components that remain suitable for re-use and directing investment at the components which will provide a meaningful improvement. The upgraded system must also operate within the constraints seen in Table 1.

Table 1

PrecisionBeam's requirements and constraints

Category	Constraint	Relevancy
Budget	$10,000 maximum for new hardware	Limits CPU, memory and storage choices; favors reuse
Power	Server under 1,000 W from the wall (UPS)	Affects all choices, mainly the processor
Space	18U free, current chassis occupies 2U, depth 700 mm	Affects the choice of the case and infrastructure, but overall, not a big limitation as plenty space is provided
Reuse	Hardware fit for purpose should be reused	Directs investment to the bottlenecks and promotes more friendly spending.
Future Expansion	Plan for detector upgrades	Forces additional research to predict possible future requirements and scale every equipment to support the possible future rate.

The objective is therefore to maximize throughput and remain within the client's physical, electrical and financial constraints, while maintaining room for future detector upgrades, possibly resulting in higher event density. Particular focus must be placed on parallel processing, which would allow for processing chunks of incoming data stream simultaneously. Consequently, the solution must not be optimized for processor performance alone, but rather balance across constraints, components and performance.

3. Current Recommendation Diagnosis

Initially the data load of the upgraded detector should be established, to proceed with identifying the design requirements. A detector of this scale can be compared with the Belle II Pixel Detector, which operates in comparable conditions. Therefore, it is assumed to produce approx. 20 GB/s of data at maximum intensity (Huber et al., 2021), where the raw rate is reduced usually by a factor of 30 by region-of-interest-filtering (built into the detector) (Geßler et al., 2015), and 10x in a different kind of detector (Adamczyk et al., 2022). This will lead to a rate of around 0.67-2 GB/s (see table 2) being sent to the computer system for filtering. Belle II was designed to keep about $1/5^{th}$ (20%) of events after filtering (Lee et al., 2012), and in practice keeps about $1/8^{th}$ (12.5%) (Itoh et al., 2020). Therefore 15% was used as the safe assumption for PrecisionBeam, giving roughly 100-300 MB/s to storage (see table 2). The filtered data is meant to be sent out to the analysis facility immediately, which manages the storage of the data.

Table 2

Estimated minimum system requirements for the upgraded detector

Case	Raw data generated by the detector (GB/s)	Reduction made by the detector's pre-processing	Data rate to computer for processing (GB/s)	Data rate to storage at 15% (MB/s)	Amount of data produced daily (TB/day)	Maximum storage needed to account for 3-day backlog for the network (TB)	Network transfer rate required to move the data immediately (MB/s)
Minimum	20	30x	0.67	100	2.88	8.64	100

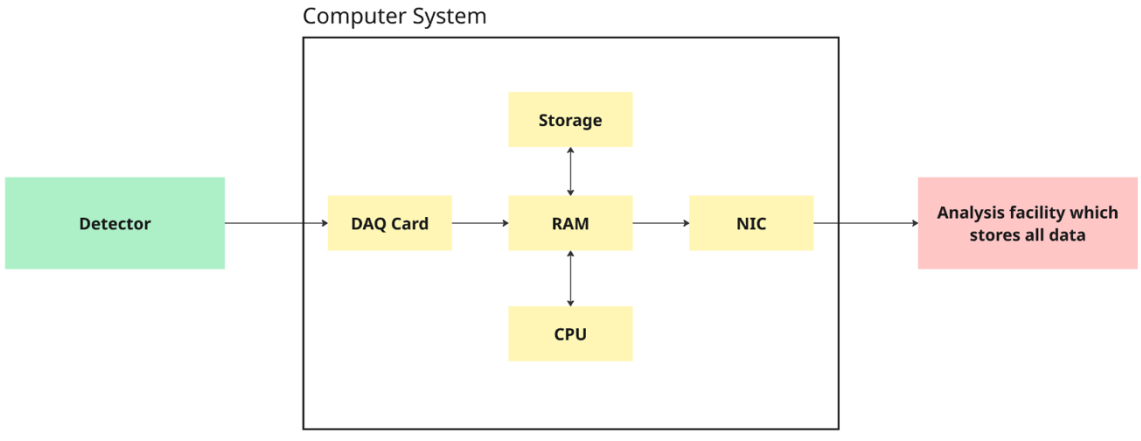
Realistic Low	20	25x	0.8	120	3.46	10.37	120
Mid	20	20x	1.0	150	4.32	12.96	150
Realistic High	20	15x	1.33	200	5.76	17.28	200
Maximum	20	10x	2.0	300	8.64	25.92	300

The 30x and 10x cases follow Geßler et al. (2015) and Adamczyk et al. (2022), despite the 10x being from a different detector (a silicone vertex detector, which operates around the pixel detector), it is used as a minimum reduction, which gives the worst-case scenario data rate, keeping the system's requirements on the safe side, whereas the other intervals were based on intervals between them. Calculation of rate at which data is sent and stored, and the column 6 calculation assumes continuous operation through 8h at full intensity. The storage backlog was decided to be 3 days, this design assumption was made to account for the network downtimes, covering a Friday evening outage over a weekend, and an extra working day to resolve the issue. Column 8 suggests a required rate to support immediate data forwarding to the analysis facility.

The flow of the data through the system can be represented in figure 1, with the current system specifications listed in table 3, and an in-depth analysis of these components against the new requirements in section 4.

Figure 1

System data flow diagram



The figure shows the data transfer, adding context to table 2.

Table 3

Current system specifications

Component	Spec	Est. Power	Description
Rack	24U 19" rack, 2U server (EIA-310), 1U patch panel, 1U switch, 2U UPS (18U free)	-	The 19-inch standard (governed by EIA-310 specifications) was globally universal in 2008 (Flower, 2025), and the most common choice nowadays as well, making it possible to be reused (Yuan, 2026).
Chassis	2U server rack – SUPERMICRO CSE-822T-400LPB (Newegg, 2008). 6 x 3.5" hot-swap SAS/SATA bays (all used), 4 x 80 mm hot-swap fans, 1 x 5.25" bay, slim DVD bay, 7 low-profile expansion slots, depth 650 mm (Super Micro Computer, n.d.-a).	Fans about 20 W	Hot-swap bays are the drive slots (hot-swap means that a failed drive can be replaced while the server keeps running), all of them will be occupied as we have 2 x 250GB drives (for OS) and 4 x 1 TB drives (for events), and the interface was supported by the chassis (SATA). The fans can also be replaced while the server runs, and they speed up as temperature rises. Slim DVD bays are spaces for removable-media drives (less common nowadays). Expansion slots are where the DAQ or RAID cards are, plugged into the motherboard to their desired spots to add a function the motherboard lacks. Low-profile suggests their size, which is because of the 2U case, standard height cards will not be supported, only low-profile cards, which are roughly half the height of a standard card. Likely this chassis will need replacing, explored in section 4.
PSU	Single 400 W (Supermicro PWS-401-2H), supplied with the chassis (Newegg, 2008), non-redundant.	Supplies up to 400 W (at current specifications about 250 W are used by components)	Some energy is lost as heat in the conversion, so the actual power used (from the wall) will be bigger than what the components actually use, depending on the efficiency. Non-redundant means that there is no backup PSU, so if it stops or fails, the computer shuts down. This could lead to data loss, integrity issues and general issues caused by the facility not producing data despite a working detector, which is something to consider during the solution in section 4.
Motherboard	Supermicro X6DHE-XG2 with 2 CPU socket, Intel E7520 chipset, 6.4 GB/s front-	Chipset take about 10 W (Intel Corporation, 2004a), and I/O hubs, NICs and	This is a 2008 server board where dual-socket board, also running single socket. The E7520 chipset manages communication between components, and the front-side bus describes the bus connecting the CPU and the chipset, which all of the processor's memory traffic goes through, with data transfer rate of 6.4 GB/s at

	side bus, 8 memory slots, 5 PCI-X slots and 1 PCIe 1.0 x8 slot, 2 SATA ports, dual GbE, VGA (Super Micro Computer, 2007)	video about 15 W	most. One socket and 4 memory slots were occupied. The expansion slots hold the RAID card in the PCI-X slots, while the newer PCIe 1.0 x8 slot holds the DAQ card. The two SATA ports connect the OS drives, and the event drives are connected through the RAID card. The dual GbE ports provide the network connection (with their own onboard NIC). There's also a VGA port for video output for a monitor (outdated, would need a HDMI adapter for a modern monitor now), 5 USB 2.0 ports (but 3 are unusable without a front-panel kit), 1 PS/2 keyboard and 1 PS/2 mouse port, and 2 serial ports (only one on the back panel, one can be accessed with the included COM2 cable). Majority of the ports are outdated, and only Nocona processors and at most 16 GB of DDR2 RAM would be supported (Super Micro Computer, 2007), so any attempt at higher scale multi-processing, with faster memory would require a different board. This is reasoned later in this section and analyzed deeper in section 4.
RAM	4 GB: 4 x 1 GB Kingston KVR400D2S4 R3/1G, DDR2-400 (PC2-3200) CL3 (Serverblink, n.d.) 4 of 8 slots free	Approximately 12-16 W	Used for holding incoming detector data while it waits to be processed. The RAM's bus clock works at 200 MHz, with DDR (double-data rate) it sends data twice per cycle, hence 400 million transfers per second (400 MT/s). PC2-3200 means the same thing, as 8 bytes are moved in each transfer, so 8 x 400 000 000 = 3.2 GB/s per module (so stick) (Hennessy & Patterson, 2019). Technically a transfer rate of 12.8 GB/s could be achieved with 4 channels, but because there are only 2 channels (1 channel for 2 sticks) the maximum would be 6.4 GB/s for this board, which is also capped by the motherboard's front-side bus speed (which sets the limit on how fast memory can be transported to the CPU), which because its 6.4 GB/s it doesn't cut the rate any further. Every piece of incoming data passes through memory several times, as initially the DAQ writes the detector data (up to 2 GB/s as in table 2) then the processor reads it(up to 2 GB/s as in table 2), then the filtered data needs to be written (up to 0.3 GB/s as in table 2) and then the written data is read out again to be sent on the network, or into the disk (up to 0.3 GB/s as in table 2). Thus 2.0 +

			2.0 + 0.3 + 0.3 = 4.6 [GB/s], at most is required, as even though memory handles these requests in turn, all of them must be completed every second. 4.6 GB/s is 72% of the 6.4 GB/s peak, leaving little headroom for bursts. Additionally, the 4 GB (reduced to 3 GB because of the OS) would mean a buffer of at most 1.5–4.5 s of incoming data at 0.67–2.0 GB/s can be achieved (3 GB ÷ 0.67 GB/s = 4.5 s and 3 GB ÷ 2.0 B/s = 1.5 s). This is below the 5–10 s buffer required and without the 10% + 8GB overhead (justified later this section) (table 4), which means it is insufficient to support the current design requirements.
CPU	Intel Xeon 3.00 GHz (Nocona), 1 core with Hyper-Threading (disabled in BIOS), 1 MB L2 cache, 800 MHz front-side bus, 64-bit capable, Socket 604 (Intel Corporation, 2004b)	About 103 W (TDP, so under heavy load) (Intel Corporation, 2004b).	3.0 GHz clock speed means the CPU completes 3 billion clock cycles per second, like all x86 processors it uses CISC (meaning that there could be fewer instructions that do more, thus one instruction can take more than 1 clock cycle, but internally it breaks those complex instructions into micro operations which imitate an internal RISC (simpler instructions, designed to take about one cycle)) (Hennessy & Patterson, 2019). This may affect the choice of the recommended CPU, as the program may only run on x86 architecture, also affect the performance, all further explored in section 4. The CPU also has 3 caches, which are small fast memories on the chip that hold recently used data and instructions, so the processor does not have to wait for the much slower RAM. It has 2 split L1s as in the Harvard architecture (fastest kind of cache), one for data (16 KB) and one for instructions, and a unified L2 (a bit slower than L1, of 1MB size) holding data and instructions together, as in a von Neumann architecture, with no L3 cache (Intel Corporation, 2004b; Hennessy & Patterson, 2019). Their size affects the speed of the CPU, as it can retrieve data it needs much faster, allowing it to focus more on processing than data retrieval, which plays a major role in choosing the new CPU (see section 4). Even though the processor supports Hyper-Threading, which lets one core switch between two threads, allowing more efficiency for programs which utilize multi-threading, it was

			found to be disabled in the BIOS. Thus, the system ran as a single thread. Enabling it would help, however, because both threads would share one core, the gain is modest. Intel engineers reported improvements of up to about 30% on server workloads (Marr et al., 2002). Assuming the data rates scale with beam intensity, this would raise the usable detector intensity from 20% to only about 26% (20% x 1.3 = 26%), still leaving the possible intensity far below detector's maximum.
OS	32-bit Scientific Linux. Per-process address space is at most 4 GB, and typically about 3 GB usable	-	A rebuild of Red Hat Enterprise Linux sponsored by Fermilab for physics computing (Fermilab & CERN, 2022). Reduces the total usable address space per process to less than 4GB, around 3 GB remain usable (Intel Corporation, 2023). This is because 32-bit means that the OS can map to 2^{32} different memory addresses, where each address points to one byte, giving 2^{32} = 4,294,967,296 bytes. Converting this into GB gives 4 GiB which is about 4GB. And with Linux, which reserves the top 1 GB of each program's address space for the operating system, about 3 GB remains for the program itself (Zijlstra, 2023). To utilize more memory (like specified in table 4), a 64-bit OS would need to be utilized, with re-installing the filtering program for the 64-bit version, which, as confirmed with PrecisionBeam, is already available.
Storage (inside chassis)	2 x 250 GB Seagate Barracuda 7200.10 (ST3250310AS) SATA HDD (RAID 1) for the OS, on the motherboard's two SATA ports 9 (Seagate Technology LLC, 2007b) plus 4 x 1 TB Seagate Barracuda	About 16 W for the OS drives and 45 W for the event drives (Seagate Technology LLC, 2007a, 2007b)	250GB reserved for OS, mirrored across 2 drives simultaneously thanks to RAID 1 architecture, so both disks hold the same data in case one fails to ensure the system is still running. They each read at up to 106 MB/s (Seagate Technology LLC, 2007b), which sustains the current system where majority of operations are made, however since the booting and loading the program requires sequential reading of big files and random reading of many small files scattered across the disk, this causes the server to boot and load significantly longer. Enterprise hard drives designed for data centers read at about two and a half times faster, at about 261 MB/s (Seagate Technology LLC, 2019), while nowadays data-center servers use SATA or NVMe SSDs to boot, the second offering about 6.5 times the throughput, (the practical speed) of SATA (Intel Corporation, 2021). Drives advised particularly for booting, such as the

	7200.11 (ST31000340 AS) SATA HDD (RAID 5) for event data connected by the RAID card (Seagate Technology LLC, 2007a)		recommended one, reads at over forty times faster, at up to 4,500 MB/s (Kingston Technology, n.d.-c) (section 4.3). There are also the event storage disks, which purpose is only to be an outage buffer, as the data is supposed to travel outside of the facility immediately after filtering is complete to be stored outside of the facility. Currently, they cannot hold the 8.64–25.92 TB 3-day backlog from Table 2. RAID 5 allows 4 drives to act as one storage. It reserves one drive's capacity for parity, giving (4 − 1) x 1 TB = 3 TB usable (Patterson et al., 1988). Additionally the rate of data writes could also be a bottleneck as each drive writes at most 105 MB/s (Seagate Technology LLC, 2007a), so with three drives worth of capacity, the array writes at most about 3 x 105 = 315 [MB/s], but only when large blocks are written at once, as small writes are penalized by RAID 5 due to having to update the parity (Patterson et al., 1988). This barely matches the 300 MB/s maximum (Table 2), leaving no headroom, and will be further analyzed and addressed in section 4.
DAQ card/bus (connects detector to the computer and transports data from one to the other)	Custom made, low-profile, PCIe 3.0 x8 DAQ card delivered with the detector, running in the motherboard's PCIe 1.0 x8 slot (Super Micro Computer, 2007)	About 25 W at most (PCI-SIG, 2013)	Connects the detector to the server and writes the event data directly into memory using DMA, without involving the CPU, as is standard for high-throughput I/O devices like the DAQ (Tanenbaum & Bos, 2015). Since the card is PCIe 3.0 supporting a transfer rate of about 8 GB/s in each direction (as PCIe 3.0: 8.0 GT/s is about 1 GB/s per lane, and x8 suggests 8 lanes → 1 x 8 = 8 GB/s), and the motherboard is PCIe 1.0 with a rate of about 2 GB/s in each direction (as PCIe 1.0: 2.5 GT/s which is about 250 MB/s per lane, and x8 suggests 8 lanes → 250 x 8 = 2000 MB/s = 2 GB/s) (Nandra, 2009), limiting the card to 2 GB/s, but it does satisfy the current system's requirements (see table 2) without headroom for data bursts. Thus, section 4 will also address that a motherboard should have the newer generation port, but the DAQ itself can be reused, hence remains outside of this investigation.
RAID card (what the event data drives are connected to,	3ware 9550SXU-8LP, 8-port SATA RAID controller,	Up to 10 W (AMCC, n.d.)	It connects and manages the four event drives, since the motherboard has only two SATA ports. Likely won't be reused as modern motherboards no longer have PCI-X slots, and usually there is enough SATA ports on the board for the drives

to act as one unified storage)	PCI-X (AMCC, n.d.)		directly, and RAID can be used through the software built into the new Linux distro (see section 4).
Network	Onboard 1 GbE, one port used, connected through a 1 GbE switch and uplink to the analysis facility (assumed)	Included in motherboard	About 118 MB/s achievable (IBM, 2026) which only meets the minimum requirements of 100 MB/s (table 2). Up to 300 MB/s is required, so a new network card (NIC) would be needed, as explored in section 4. The switch and uplink would need to support the same speed, but the wider network is outside the scope of this report, although it will be mentioned further in section 4.
Graphics	None; onboard 2D video only (Super Micro Computer, 2007)	Included in motherboard	Not needed, as the filtering program does not use graphics processing. The onboard video is enough for a monitor during setup, also in the improved version.
Ports and I/O (only the currently usable, those built into the motherboard, but currently not accessible are skipped)	1 x VGA, 2 x USB 2.0, 1 x PS/2 keyboard, 1 x PS/2 mouse, 1 x serial, 2 x Gigabit Ethernet (Super Micro Computer, 2007)	Negligible when idle. About 10 W in use (DVD drive estimated to about 2–5 W while reading, and up to 2.5 W per USB port, thus 2.5 + 2.5 + 5 = 10) (USB Implementers Forum, 2000). Ethernet included in motherboard	One Ethernet port is in constant use, connecting the server to the switch to send the filtered data (see Network) and the second is unused. The other ports are empty by default, as the server runs without a screen or keyboard. These are only plugged in for setup or maintenance: a keyboard and mouse draw about 0.5 W each from the server, while a monitor plugs into its own power socket, so it adds nothing to the server's power. The USB ports can also be used for an external drive, but USB 2.0 runs at 480 Mb/s (60 MB/s) at most (USB Implementers Forum, 2000), so copying the full 3 TB array would take at least about 14 hours, making it unsuitable for retrieving event data.
UPS	2U rack UPS, about 1000 W	-	Prevents the rack from failing during short power cuts, allowing a safe shutdown. Provided the new system's power stays within its rating it will likely be re-used (see section 4). Like the DAQ, it stays outside of the scope of this investigation.

Information in the context column, if not supported by their own citation, were taken from the source cited in the row.

The facility stated that during running the detector at highest intensity, CPU and RAM were held at maximum utilization, causing overheating, which led them to be forced to decrease the intensity. Previously they were recommended to increase RAM, as it was assumed that the system lacks space to hold incoming data. This assumption is

considered wrong, because memory acts only as a buffer between the arrival of detector data and its processing, storing data which waits for processing, where a lack of RAM would show itself through low CPU utilization, and a high disk activity (Silberschatz et al., 2018). Since, when memory is exhausted, the OS starts swapping data into storage (thrashing, a continuous loop of page faults, where the system starts spending more time swapping data from secondary storage into memory than executing the program) (Tanenbaum & Bos, 2015), making the CPU spend more time waiting for data than executing tasks, since a disk access takes milliseconds compared with roughly 100 nanoseconds for RAM (Hennessy & Patterson, 2019).

Processor fully utilized would indicates it doesn't catch up with the incoming data rate, and memory fills as a consequence, growing by the difference between the data processing and arrival rate, eventually exhausting it. Currently the system handles only about 20% beam intensity, assuming the data rate scales with intensity, the current processor can handle roughly 20% of the 0.67–2.0 GB/s produced at full intensity (table 2), hence about, about 5 times too slow, with Hyper-Threading enabled, 4 (table 3).

Nevertheless, the recommendation wasn't all wrong. The current RAM needed is approximated from the rates in table 2, with a buffer estimated based on the reference detector, common connection conditions and interruptions, and additional overhead accounting for system processes and data fluctuations.

5-10 second buffer time is a design assumption made to cover for network delays and bursts, since using the system at full utilization is undesired, as delay grows sharply as a system approaches full utilization (Vishnu, 2025). The lower bound is taken from the reference detector's (pixel detector in Belle II) data reduction system, which buffered the incoming data for up to 5 seconds while the high-level trigger decided which events to keep (Geßler et al., 2015), making it an estimation aligned with common practice for this kind of detector. The upper bound covers short interruptions in the output side, the network. If it briefly stops delivering data, the protocol used to send it (TCP), waits before retransmitting, starting at about one second and doubling with each failed attempt (Paxson et al., 2011), so three failed attempts can be estimated to take 1 + 2 + 4 = 7 s. A 10 s buffer is a reasonable estimate to account for a burst combined with a network interruption or a storage stall without losing data. Additionally, longer interruptions are not memory's responsibility, because if the network is down for minutes or days, filtered data is written to the local disks instead, which are sized for a three-day backlog (table 2). Memory therefore covers seconds, and storage covers hours to days.

There is also an extra overhead of 10%, which represents additional space for fluctuations and a fixed 8GB added for the OS, the filtering program and additional buffers. Since AlmaLinux 9 (justified in section 4.4) (a rebuild of Red Hat Enterprise Linux 9 which requires about 1.5 GiB of memory, about 1.6 GB) was chosen, 1.6 GB was required for it to work (Red Hat, n.d.-a). The rest (approx. 6 GB) is for the filtering program and the kernel page cache that holds filtered data before it is written out to the disk or the network. This ensures Linux doesn't block the writing process, which happens when unwritten data exceeds 20% of available memory (SUSE, 2015).

Table 4

Minimum Required Memory Estimation

Incoming Rate	5s buffer (GB)	5s + 10% + 8GB (GB)	10s buffer (GB)	10s + 10% + 8GB (GB)
0.67GB/s	3.4	11.7	6.7	15.4
2.0GB/s	10	19	20	30

Calculated through: (rate x buffer time) x 1.1 + 8 GB, for example (2.0 GB/s x 10 s) x 1.1 + 8 GB = 30 GB.

The system would therefore require about 12-30 GB of RAM, making it far above the current 4 GB RAM and 3 GB per-process limit of the 32-bit OS (table 3). Thus, the recommendation was right to assume that more RAM is generally required. However, other components of the system like the transfer capacity of DAQ, the storage or network, despite not showing at first glance, also contain bottlenecks which would cause stalling and memory exhaustion, or in the case of storage space, data loss. The analysis of each of them against the requirements and improvement ideas are explored in section 4.

4. Proposed System Specifications

4.1 Processor

As the processor was identified as the root cause behind the computer system in section 3, its replacement is unavoidable. Section 3 established that the new processor

needs about 5 times the throughput of the previous one. However, this is only the estimated minimum for the average system requirements (see table 2). The processor is sized for at least 10 times the throughput for three reasons listed in table 5.

Table 5

Reasons behind the bigger processor requirements

Reason	Explanation
System tasks	The cores are not used by the filtering program alone. They also need to handle operating system tasks such as interrupts from the DAQ card and network, polling, sending data over the network, and the parity calculations of the software managing drives (Section 4.3 and 4.7) (Tanenbaum & Bos, 2015). At full intensity, the DAQ card has a higher data rate, thus more filtered data must be sent and stored. More data rate means more of the processor's time goes to these tasks, as they grow with the amount of data rather than staying fixed. There is also additional overhead caused by running the same set of filtering operations on each core and coordinating the program across many cores. Thus, part of the processor's capacity must be reserved for this work.
Bursts and full utilization	The detector may not have a consistent data rate, as events arrive asynchronously and unpredictably (Wielers, 2014), which sometimes surpasses the average maximum, and sometimes barely reaches it, which wasn't accounted for in the average system requirements (table 2). Considering, delay grows sharply as a system approaches full utilization (Vishnu, 2025), which degrades performance, sizing processor cores to exactly 5 times the throughput, meaning full utilization at the maximum rate, it would be undesired. Also, any burst going over the maximum rate would build a backlog, which at stable maximum rate, could hardly be cleared. Overloading the CPU is undesired, as it slows itself down in order to prevent overheating and stay within its power limit, thus is generally undesirable (Schuchart et al., 2016). At 10x, it runs at about 50% utilization at the maximum rate, leaving capacity to clear bursts and maintaining a utilization-friendly approach.
Future growth	Since the last detector upgrade substantially increase the event rate and the facility keeps hardware for many years, a further detector upgrade during the new system's lifetime is likely. Additionally, one of the requirements is to account for possible future expansions. Therefore, the system should account for a

	possible future improvement. Since the size of the next upgrade cannot be known in advance, it is assumed that PrecisionBeam's maximum incoming rate could double, which gives the system margin for future growth. This would made the incoming data rate result in 1.34-4 GB/s (from table 2) still supported by the DAQ (see table 3). The current processor handles about 20% of the current rate, roughly 0.13–0.4 GB/s (section3). Thus, 1.34 ÷ 0.13 is about 10.31, and 4 ÷ 0.4 is 10. Therefore, at those conditions the required throughput would need to be 10 times the current throughput, instead of the previously estimated 5 (section 3).

The current single-core processor (table 3) handles events one at a time, utilizing only instruction-level parallelism within a single instruction stream (Hennessy & Patterson, 2019), meaning that it can only pipeline instructions to ensure all components of the CPU (like ALU, registers, etc.) are continuously occupied, every cycle. Making a single-core faster wouldn't solve the problem, as it would mean the clock rate would need to be about 30 GHz, instead of the current 3.0 GHz, which is not achievable as clock speeds reached a ceiling in the mid-2000s, as power consumption rose steeply with clock rate and manufacturers have started developing processors with more cores and bigger caches (Hennessy & Patterson, 2019). This is possible as with development, more transistors can fit on a chip, roughly doubles every two years as predicted by Moore's Law (Moore, 1965), which was observed as more efficient in increasing core counts, rather than the clock rate, where parallel processing brings more gain than a faster single core. Modern cores do complete far more work per cycle as in table 6 (see clock rate), however with the recommended processor it is only 5x. Thus, the required throughput must come from processing many events in parallel across multiple cores and utilizing the filtering program's multithreading support.

Each event passes through the same steps: a checksum checks to discard corrupted data inside events, a threshold cut that discards weak readings, and a count check that rejects events with too little readings remaining. The 15% kept events (table 2) are written into a 1.5 MiB output buffer (about 1.57 MB), that the program allocates for each core, which is handed over to be written out once filled. Since each step depends only on the event being processed, events are independent of each other, thus, can be divided across cores and filtered at the same time, as in Belle II (Itoh et al., 2020). According to Amdahl's law the speedup from parallel execution is limited by the part of a program that must run serially (Amdahl, 1967), which, in this scenario is limited mainly to distributing the incoming data to threads and writing processed results, since

reading input doesn’t require CPU, as the DAQ card’s writes, of incoming data, to memory are handled by its DMA (Tanenbaum & Bos, 2015).

Thus, the consideration is between using a multi-core CPU or combining many CPUs, with modern technology, first option ranks much cheaper and energy efficient (ServeTheHome, 2026; Dell Technologies, 2026). Therefore, the processor is replaced with the Intel Xeon Gold 5512U, a 28-core server processor suited to splitting a continuous data stream across many cores. To justify this choice, first the current and recommended processors are compared in table 6.

Table 6

Comparison of the current and recommended CPU

CPU Specification	Current CPU (Intel Corporation, 2004b)	Recommended CPU (Intel Corporation, n.d.-c)	Context (Intel Corporation, 2004b; n.d.-c)
Architecture	x86 (CISC), NetBurst	x86-64 (CISC), Raptor Cove cores	NetBurst was Intel's design of the early 2000s, which consisted of using a very long instruction pipeline to reach high clock speeds where, in order to achieve those fast clock speeds, each instruction is passed through many short pipeline stages (about 20, some up to even 30 plus), thus when an pipeline flush occurred due to for example a control hazard (the wrong instruction was loaded as the previous one changed the flow), far more cycles were wasted with that many stages, overall making each cycle less productive with less instructions per cycle performed. Raptor Cove is a modern core design, which completes more instructions per cycle. Both use x86, CISC (Complex Instruction Set Computer) where, fewer, more complex instructions each do more work, while RISC (Reduced Instruction Set Computer) (its opposite) uses more simpler instructions. Nowadays, x86 processors break complex instructions into simpler micro-operations (utilizing a RISC-like micro-operations behind a CISC), so the two now perform similarly (Hennessy & Patterson, 2019). x86 is retained because the filtering program and the DAQ card's driver are built for x86 and cannot be assumed to run on ARM without additional modification, and ARM's main advantage is lower power per core, since the system's power is well within the client's limits (Table 1), it matters less here than compatibility.

Cores/Threads	1 / 1 (Hyper-Threading disabled) 1 / 2 (with Hyper-Threading	28 / 56	A core is an independent processing unit that runs its own stream of instructions, and even with Hyper-Threading the current CPU reaches only about 1.3 times its throughput (table 3). On multiple cores, independent tasks can be performed in parallel, like filtering independent events. The 28 cores supporting hyper-threading, will not equal to 56 cores, as hyper threading works by switching between 2 threads, while one for example waits for data, meaning that it doesn't double the work a core can do, rather fills gaps. Intel themselves claimed a gain of up to around 30% (Marr et al., 2002), and a newer study reported only about 13–18% for particle-physics software on modern servers (Schnepf et al., 2024), so the calculation below uses 28 physical cores, with Hyper-Threading treated as extra headroom. The number of cores was chosen to give about ten times the current throughput, twice the five-fold minimum, as calculated below.
Process node	90 nm	Intel 7	Process node illustrates the size of the smallest features on the chip, where the smaller it is, the more transistors can fit in the same area (Moore, 1965), making more fit on one chip within a reasonable power budget. The new CPU, compared to the current, shrunk from 90 nm to Intel 7, which is what makes it be able to fit 28 cores and larger caches.
Clock rate	3.0 GHz	2.1 GHz (3.7 GHz in boost mode on a single core, and 3.0 GHz with all cores boosted)	Clock rate describes the number of cycles a core completes per second (Hennessy & Patterson, 2019). One clock cycle is one tick of the processor's clock. Each instruction passes through several steps like fetch it, decoding it, executing the operation, and writing the result to memory if required. These steps can take one or more cycles, and in modern architecture, instructions are pipelined, where the stages of different instructions overlap, thus a new instruction can start before the previous one is finished, as they would use different hardware. Complex instructions, especially in CISC architectures, may occupy more than one cycle per instruction on average (Hennessy & Patterson, 2019). Clock rate is used to estimate the speed at which a processor can execute instructions, but with different architectures it isn't an unequivocal measurement (further explored during core number estimation, later this section). Boost mode (Intel Turbo Boost) raises the clock above the base rate when power and temperature allow and when its needed, usually in short bursts of extra speed. Under sustained full load, all 28 cores are only guaranteed the base rate, hence 2.1 GHz is the figure used for

			planning, but all cores can usually run at up to 3.0 GHz (Super Micro Computer, n.d.-b). A lower base clock is the trade-off for 28 cores: power rises steeply with clock speed, so running many cores slightly slower delivers far more total work within the same power budget (Hennessy & Patterson, 2019). But due to the architecture differences, where a modern CISC core is designed to do more per cycle than NetBurst, a lower clock does not mean a slower core. In single-thread benchmarks (so comparison of work processing power of one core working on one thread), the current processor scores 607 (PassMark Software, n.d.-a), and the recommended one about 3,019 (PassMark Software, n.d.-b), about 5 times as fast, which on its own would not meet the 10x target (see above)
L1 cache	Private split: 16 KB for data and 12K micro-operations for instruction side	Private split: 48 KB for data and 32 KB for instructions per core	L1 is the fastest cache level available, and the smallest one. Here its split into separate data and instruction caches, as in a Harvard architecture. At this level, during single-core instruction pipelining, the core often needs an instruction and its data in the same cycle, where separate caches let it fetch both at once instead of making one wait for the other (Hennessy & Patterson, 2019). Also, the new L1 data cache is three times larger, allowing more of each event being filtered stays in the fastest cache, or frequently accessed instructions like for the filtering program, reducing memory access time.
L2 cache	1 MB	Private unified 2MB per core (56 MB total)	L2 is slower than L1 and usually a bit bigger. Here it's unified and private, holding data and instructions together for each core exclusively, as in von Neumann architecture, mainly because in larger sizes, a unified cache uses its space more efficiently, since as the split between code and data can change according to a program's needs (Hennessy & Patterson, 2019). This level is key for the filtering program, as each core's 1.57 MB output buffer is written to repeatedly, together with the event it currently works on. Assuming a data transfer rate similar to the one of the reference pixel detector, which has a trigger rate of 30kHz, where each trigger is an event (Huber et al., 2021), so 30,000 events per second, would mean each event is about 22-67 kB (0.67–2.0 GB/s ÷ 30,000, using table 2), so each core works on about 1.6 MB (1.57 + 0.067 MB). The current 1 MB L2 cannot hold this, while the new 2 MB L2 per core can.
L3 cache	None	52.5 MB shared unified	L3 is the slowest cache level in this architecture but also the biggest, which is the last memory level before RAM, and it is also unified (von

			Neumann) (Hennessy & Patterson, 2019). On the current CPU, everything that isn't in L2 goes straight to RAM to be fetched, which takes much longer than fetching from caches (Hennessy & Patterson, 2019), whereas the new CPU's 52.5 MB L3 may catch many of those misses.
Memory Communication	No memory controller, CPU reaches memory through motherboard's CPU exclusive bus, the front-side bus (6.4 GB/s, thus the chipset controls 2 channels of DDR2-400 (Super Micro Computer, 2007).	Memory controller built into the CPU: up to 8 channels of DDR5-4800 connected directly, up to 8 x 38.4 GB/s = 307.2 GB/s (calculated) (the maximum the CPU can support, but the actual number depends on the specific RAM,how many modules (so channels) are used and how many the motherboard will support, but the CPU has 8 channels)	The memory controller is responsible for reading and writing data in RAM (no matter by who, the CPU or components, it manages it all). On the current system, it is in the chipset, thus every request from the CPU travels over the shared front-side bus and back, adding both an additional rate limit. On the new system, it is on the CPU, which talks to RAM directly through its own channels, which other components also use, removing the front-side bus bottleneck entirely. An abstracted visualization can be seen in figure 2. Even with 1 RAM stick the roughly 4.6 GB/s of memory traffic required (section 3) is covered. Hence, memory bandwidth will not limit the new system.
PCIe	No PCIe in the CPU: data travels from the DAQ card through the PCIe lane to the chipset limited by the PCIe 1.0 x8 lane's capacity (2 GB/s) (see section 3 table 3 DAQ card)	80 PCIe 5.0 lanes built into the CPU, about 3.9 GB/s per lane in each direction (Keysight Technologies, 2020).	PCIe (PCI Express) connects expansion cards, such as the DAQ card in this scenario, through lanes (x8, suggest 8 lanes or x16, 16). In the current system, there are 8 lanes, and it is connected to the chipset, running first-generation speed, whereas in the new system, the lanes come straight from the CPU, and as PCIe is backward compatible, the PCIe 3.0 x8 DAQ card can run at its full speed of about 8 GB/s (Nandra, 2009), four times the 2 GB/s maximum incoming rate (table 2). The 80 lanes also leave room for future cards, and ensure the data rate transfer fulfills the requirement (table 1) and leaves headroom to account for data bursts, fluctuations, and future expansion as mentioned in section 3.
TDP	103 W	185 W	TDP (thermal design power) is the heat (so energy) a processor produces under sustained heavy load, which the cooling system must remove, making it a good estimate of the power drawn by the CPU at full load (Intel Corporation, n.d.-c). The new CPU draws more in total, but it

			is a reasonable sacrifice for the processing, considering it fits within the client's limit (table 1)
Price	-	About $1476.00 (Intel Corporation, n.d.-c)	

Figure 2

Abstract Diagram of Current and New Systems' Communication

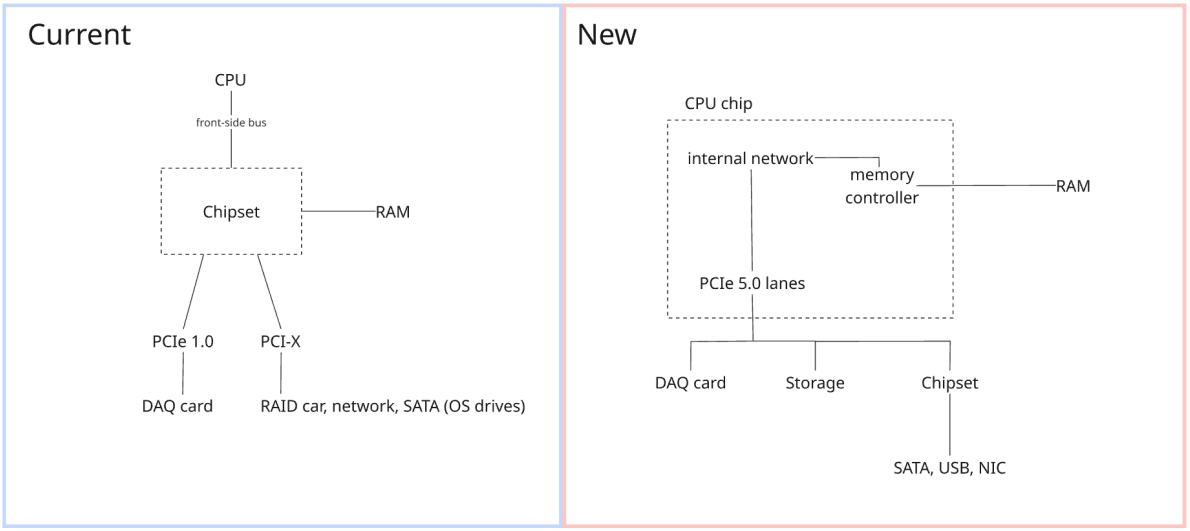


Diagram made to visualize the way memory is connected in the current and the new, proposed system.

Testing of whether processing rate scales with the number of cores, a benchmark program is constructed in sections 7-8, imitating the filtering program and cores through threads. First, however, the number of cores needed had to be estimated.

As mentioned in section 3, in parallel programming there are operations that are serial, and need to be shared across all cores, like in this scenario, allocating each core its set of events and collecting the filtered events to be saved in a shared data structure (McKenney, 2025). In result, this could create overhead and slow down the realistic processing power, as some cores would be done faster than others, leaving them idle. This will be accounted for during the speedup estimation, and the potential serial fraction is measured on a simulated filtering program in section 7-8.

Amdahl's law predicts how much faster a fixed amount of work can be completed when it is divided between more cores, given that part of the work cannot be divided (serial tasks) (Amdahl, 1967). Given that we have the approximate number of how faster the system would need to be, it can be used to identify how many cores would be needed. The equation is the following:

$$Speedup = 1 \div ((1 - P) + P \div N) \quad (1)$$

Where:

P is the proportion of the filtering program that can run in parallel (%)

N is the number of cores

Thus, the law also shows the limit of adding cores. As the number of cores grows, the parallel part shrinks towards zero, but the serial part does not, so the speedup can never exceed 1 ÷ (1 − P), however many cores are added. Making Amdahl's law suitable for estimating how many cores are needed and also for showing when adding more stops bringing meaningful improvement, as each additional core brings a smaller gain as the ceiling is approached.

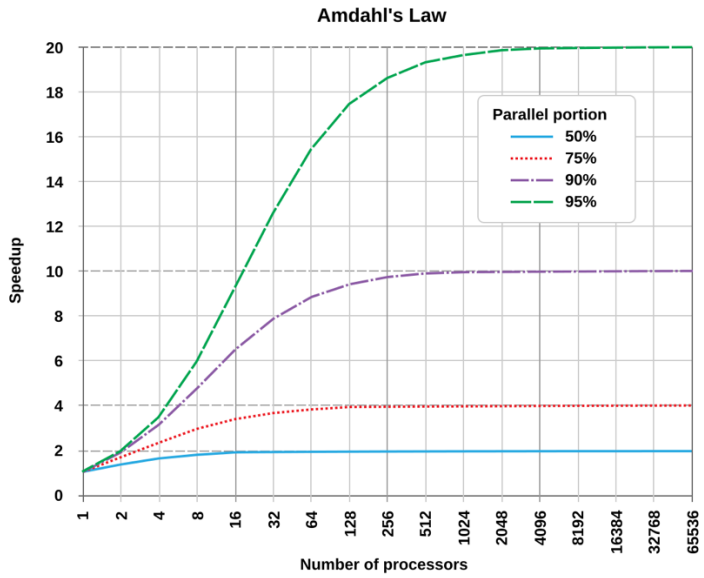
In this case, P doesn't need to account for system tasks like handling interrupts and sending data because they have already been accounted for in the 10x assumption (see table 5x). Most of the filtering program consists of tasks which can be done in parallel (as stated in section 3), as they consist of the checksum check and threshold cut that processes every byte, so does reading of every event and saving it to the core's cache (inside a buffer), thus they make up almost all of the work. The serial tasks, distributing events as they arrive and handing over full buffers to RAM, involve a small, usually fixed amount of work each, as the overhead of writing incoming data to RAM is handled by the DAQ by its DMA (Tanenbaum & Bos, 2015), and buffers are passed on without copying. PrecisionBeam estimated the filtering program to have a serial part that makes up about 1% of the work, a figure typically measured by counting the instructions executed in the program's parallel and serial parts, which aligns with the characteristics of the program.

However, the estimate assumes the work divides perfectly between cores. Real systems rarely achieve this because firstly, threads must coordinate when taking events where realistic events vary in size, and secondly, the operating system sometimes interrupts threads, both causing delays in some cores while others wait (Herlihy et al., 2021; Tanenbaum & Bos, 2015). P is therefore assumed to be 0.95, treating 5% of the work as serial, as it leaves a margin of 4% above the documented 1% to cover these overheads.

With P = 0.95, the maximum possible speedup is 20 times (1 ÷ (1 − 0.95) = 20), however many cores are added. The 10x target is half of this ceiling, so it can be achieved with a realistic number of cores, as seen in figure 3, but if the serial portion was 10% or more, this wouldn't be achievable with adjusting the core count (one processor or several, could have met the target but only by reducing the serial part of the program by increasing clock rate or the processing hardware number). Amdahl's law also assumes the before and after throughput/cores have the same clock rate. But as explored in 6, the recommended CPU could be up to 5 times as fast, assuming equal speed is therefore conservative.

Figure 3

Amdahl's Ceilings Graph



From Wikipedia contributors (2026)

Furthermore, the same equation (1) is used to calculate the required number of cores. N is usually taken as the number of physical cores rather than threads, even for cores with Hyper-Threading, which lets each core switch between two threads, as it does not double its capacity, and for particle-physics software on modern servers, it adds about 13–18% (Schnepf et al., 2024). Gain coming from Hyper-Threading is therefore treated as additional headroom.

Estimated minimum number of cores at target 10x:

$$Speedup \geq 10$$

$$1 \div (0.05 + 0.95 \div N) \geq 10$$

$$0.95 \div N \leq 0.05$$

$$N \geq 19, so\ at\ least\ 19\ cores$$

Since server processors come in fixed core counts, the next would be a 20, 22 or a 24 core, compared in table 7.

Table 7

Predicted speedup by core count and parallel proportion (Amdahl, 1967)

Core count	Parallel proportion	Predicted speedup	Meets 5x minimum?	Meets 10x target?
20	0.99	16.8x	yes	yes
	0.95	10.3x		yes
	0.94	9.3x		no
22	0.99	18.2x		yes
	0.95	10.7x		yes
	0.94	9.7x		no
24	0.99	19.5x		yes
	0.95	11.2x		yes
	0.94	10.1x		yes
28	0.99	22.0x		yes
	0.95	11.9x		yes
	0.94	10.7x		yes

A 20 would give about 10.3 times speedup, meeting the target by only about 3%, so a serial part just one percentage point larger than assumed (P = 0.94) would reduce it to about 9.3 times, and even a 22 would give about 9.7 times at P = 0.94, below the target. Whereas the 24 still reaches the target at P = 0.94 (1 ÷ (0.06 + 0.94 ÷ 24) = 10.1), providing a margin against the uncertainty in the estimate of P. Even if the serial part was more than three times larger than assumed (P = 0.835), the 24 would still meet the 5x minimum.

Despite 19 cores being the mathematical minimum for the 10x target, 24-core becomes the minimum, if the serial part is slightly larger than assumed, thus, the goal was to choose one that's at least 24 cores. From the processors meeting this, the 28-core Intel Xeon Gold 5512U was chosen. At a recommended price of $1476.00 (Intel Corporation, n.d.-c), it costs less than the 24-core Xeon Silver 4516Y+ ($1,554–1,564; Intel Corporation, n.d.-a) and offers four more cores, raising the predicted speedup to about 11.9x (1 ÷ (0.05 + 0.95 ÷ 28) = 11.9). The higher the cores, the less the increase in

number of cores will translate to an increase in speedup (Amdahl, 1967), therefore this should be looked out for, however due to the price difference, it was considered as the right decision.

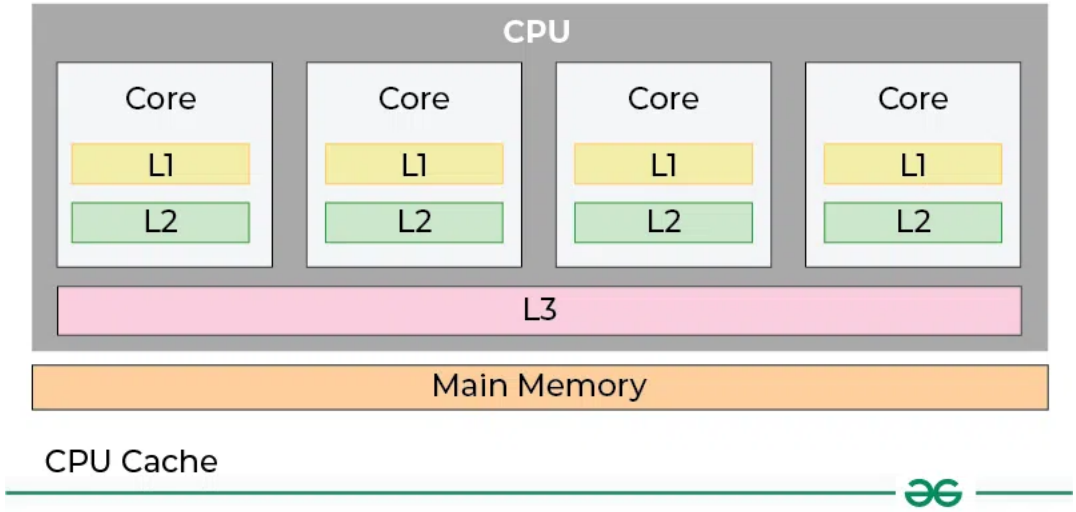
The Xeon Gold range was chosen over the Xeon Silver range because of the 5512U's single-socket design, which supports only one processor and disables the links used to connect a second one. As the facility doesn't anticipate any rapid changes in the foreseeable future, and the previous detector change took them almost 20 years, it can be concluded that they keep hardware for many years. The extra cores provide margin designed for its long service life, beyond the doubling already accounting for a detector upgrade. The 5512U also has fewer of the built-in accelerators found in other models; however, because the filtering program doesn't use these accelerators, the client would gain nothing from paying for them. This explains why the 28-core 5512U costs less than the 24-core Xeon Silver 4516Y+ while keeping the 2 MB of L2 cache per core required by the filtering program (see the cache analysis below).

This was the main rationale behind choosing the CPU, but the other features from table 6 were also considered before making the final decision.

The recommended processor's caches are larger at every level as seen from table 6. The incoming data is read only once, so caching it gives no benefit, and it depends on memory bandwidth instead (Drepper, 2007). However, as specified in table 6, the filtering program allocates a 1.6 MB output buffer for each core, that is written to repeatedly, thus, it benefits from being exclusive to each core and staying as close to the core as possible (Hennessy & Patterson, 2019), which makes L2 matter most for the filtering program. This is because it is the only cache both large enough and private to each core. The L1 cache is closer, thus faster, but, at 48 KB, far too small for 1.6 MB. The L3 cache could hold all 28 pre-allocated, by the program, buffers (28 x 1.6 MB = 44.8 MB of 52.5 MB), but it is about four times slower than L2 (in an example used during AMAT calculations later this section). It is also shared by all 28 cores and the incoming data stream; thus, the buffers would nearly fill it, compete for space and could be pushed out by each other (Hennessy & Patterson, 2019). Simplified diagram of the cache architecture can be seen in figure 4, showcasing their position relative to each core, without an accurate representation of their architecture or size. If some buffers wouldn't fit into the close by caches (L1 or L2), it would cause the processing time to increase, as data writes would be made directly to the slower cache (L3) or the much slower RAM, when L3 had pushed it out (Drepper, 2007).

Figure 4

Simplified cache organization diagram



From GeeksforGeeks (2025).

The improvement coming from allowing the output buffers and currently processed event to be stored in L2, reducing the number of relatively slower trips to L3 and RAM, can be estimated using the average memory access time (AMAT), which combines the time to find data in each cache level with the cost of missing it (Hennessy & Patterson, 2019):

$$AMAT = Hit\ time(L1) + Miss\ rate(L1) \times (Hit\ time(L2) + Miss\ rate(L2) \times (Hit\ time(L3) + Miss\ rate(L3) \times Memory\ time)) \quad (2)$$

Where:

Hit time is the time to retrieve data from a memory level when it is found there

Miss rate is the proportion of accesses for which the data is not found at a memory level, so it must be fetched from the next level

Memory time is the time to fetch data from main memory

Every access needs to at least pay the L1 hit time which is the time it takes to retrieve data from L1 cache. If an L1 miss occurs, meaning that the data or instruction that was searched in that cache to be retrieved to processor wasn't resolved, the search will move to the next level, and so on until it reaches memory (or if it isn't inside the

memory, it will be notified right away, as during resolving the virtual memory address from the TLB in cache (or page table in RAM during a TLB miss), those addresses that are currently not in memory are marked, and there would be a page fault, taking much more time than retrieving data from any of the primary memories, which will be further explained in section 4.2). Every memory miss would have its own time addition; thus, the accesses that miss L1 also pay the L2 hit time, those that also miss L2 pay the L3 hit time, and so on down to main memory. The number of accesses that miss L1 is the L1's miss rate, same principle applying for the other levels.

With the aim being to observe the approximate improvement of having a cache for the output buffer and current event, versus using L3 or RAM for the program's output buffer, rough estimates can be used. The access times are taken from Intel's Sapphire Rapids server processors, which is the predecessor of the recommended processor, where L2 has a hit time of 16 cycles, L3 hit time of about 33 ns and a main-memory access time of about 107 ns (Lam, 2023). The L1 hit time is taken as about 5 cycles. After converting the cycle counts according to each processor's clock speed, the current processor (3.0 GHz) has about 1.7 ns (5 ÷ 3.0) for L1 and 5.3 ns (16 ÷ 3.0) for L2, and the recommended processor has about 2.4 ns (5 ÷ 2.1) and 7.6 ns (16 ÷ 2.1). These same cycle counts and memory time is used for the current processor, despite its older caches and DDR2 memory being slower than these, which makes the comparison more conservative and in favor of the current CPU.

Miss rates are estimated from the 1.6 MB each core works on, where each hit rate is taken by the level's size divided by 1.6 MB. When a level is big enough for the division output to be above 100%, a small value miss rate is used rather than assumed to be zero, as some misses always occur, for example when new data is loaded for the first time (Hennessy & Patterson, 2019), happening frequently as each core constantly reads new data. Additionally, this method assumes data is only found in a cache if all of it fits there, ignoring that the commonly processor load data in the 64-byte blocks, hence reading through memory in order would miss only once per block, reducing the miss rate, as not only the desired by CPU cell would be saved into the cache, but also the surrounding ones (Drepper, 2007). The miss rates in Table 8 are therefore worst-case estimates.

Table 8

Average Memory Access Time Calculation

CPU	L1 miss rate	L2 miss rate	L3 miss rate	AMAT
Current: L1 16 KB L2 1 MB L3 none	99% (16 KB ÷ 1600 KB = 1% hit)	37.5% (1 MB ÷ 1.6 MB = 62.5% hit)	-	1.7 + 0.99 x (5.3 + 0.375 x 107) ≈ 46.7 ns
Recommended: L1 48 KB L2 2 MB L3 52.5 MB	97% (48 KB ÷ 1600 KB = 3% hit)	5% (2 MB ÷ 1.6 MB = 125% hit, meaning all data fits so it's never a miss. But instead of assuming miss rate at 0, we assume 5% for compulsory misses on new events)	10% (all data fits, but instead with 10% because despite misses on new events it also shares misses on multiple cores and one core's data may push another ones data out)	2.4 + 0.97 x (7.6 + 0.05 x (33 + 0.10 x 107)) ≈ 11.9 ns

For the recommended processor, almost every access is served by L2, and only rare misses reach L3 or main memory. Its average memory access time is therefore lower by about 3.9 times (46.7 ÷ 11.9), making it that much faster in retrieving data than the current CPU, from the cache change alone, before any gain from the additional cores.

Followed by the cache the connection to primary memory also plays a key role in the performance. The current processor reaches memory through the front-side bus, limited to 6.4 GB/s (table 3). The recommended processor has its memory controller built in, connecting directly to up to eight DDR5-4800 channels, a maximum of 8 x 38.4 = 307.2 GB/s (see table 6), far above the roughly 4.6 GB/s of memory traffic required (section 3, table 3). The bandwidth actually achieved depends on the memory installed, as each channel without a RAM stick adds nothing, and slower modules lower the rate

per channel. Nevertheless, even a few populated channels exceed the bandwidth required (explored in section 4.2). Furthermore, there are also 80 PCIe 5.0 lanes also connect expansion cards directly to the processor, so the reused PCIe 3.0 x8 DAQ card runs at its full speed of about 8 GB/s (Nandra, 2009), unlike before, where it was capped due to an older generation PCIe and reaching the maximum average rate required (2 GB/s, see table 2 and 3). Now becoming four times the previous, with spare lanes to future cards.

Then the proposed processor's 185 W TDP is higher in total than the current 103 W but shared across 28 cores it is about 6.6 W per core (185 ÷ 28), so it delivers far more work per watt and remains within the client's power limit (table 1).

Overall, the recommended CPU addresses every limitation identified in Section 3. It retains the x86 architecture; hence, the existing filtering program and DAQ driver run unchanged, while the 28 modern cores give a predicted speedup of about 12 times. The larger caches also make memory access about 3.9 times faster, whereas the built-in memory controller and newer generation PCIe lanes remove the bottlenecks surrounding the processing cycle. This removes the root cause behind the facility running at 20% intensity, while the spare capacity in cores, memory channels and PCIe lanes leave room for future detector upgrades.

This recommended processor was also compared with a cheaper alternative from the same intel line that fits the same motherboard (see table 9) at the final stage of making a decision.

Table 9

Comparison of the recommended and alternative CPU

Specification	Intel Xeon Gold 5521U (Intel Corporation, n.d.-c) (recommended)	Intel Xeon Gold 5412U (Intel Corporation, n.d.-b) (alternative)	Impact
Generation	5th Gen (Emerald Rapids)	4th Gen (Sapphire Rapids)	Newer is refined, practical difference is small, but it does stay

			supported and available longer
Socket and motherboard	LGA4677 / X13SEI-TF	LGA4677 / X13SEI-TF	No difference
Cores and threads	28 / 56	24 / 48	Predicted speedup is approximately the same, 11.9x against 11.2x
Clock rate	2.1 GHz base rate 3.7 GHz boost mode 3.0 GHz boost mode on all cores simultaneously	2.1 GHz base rate 3.9 GHz boost mode 2.9 GHz boost mode on all cores simultaneously	No significant difference
L2 and L3 (as those interest us most for the filtering program)	2 MB and 52.5 MB	2 MB and 52.5 MB	No difference, sufficient for the 1.6 MB per core
Memory, PCIe and TDP	8 channels of DDR5-4800 80 PCIe 5.0 185 W	8 channels of DDR5-4800 80 PCIe 5.0 185 W	No difference, far exceeding the requirements
Recommended Customer Price	About $1476.00	About $1113.00	Alternative about $350 cheaper

Both CPUs use the same motherboard and power; hold the 1.6 MB each core works on in L2; meet the 10x speedup target and are from the same intel line; thus, the 24-core alternative would also be a suitable choice. The 5512U was preferred as it provides four more cores, raising the predicted speedup from 11.2 to 11.9 times, has a larger L3 cache, and is a newer generation, meaning it will remain available and supported for longer, which suits this scenario, where the facility keeps its hardware for many years. The price change isn't enormous, looking at the whole budget, provided its advantages, which is what justifies this selection. However, the 5412U still remains a reasonable lower-cost option if the budget was tighter.

Initially AMD processors were also considered as they share the same architecture and are a direct competitor to Intel. However, AMD doesn't provide processors with core exclusive (private) caches of more than 1 MB, thus not big enough to sustain the programs and system needs (Lam, 2022) of approx. 1.6 MB. Part of each buffer would

then have to be stored in the slower, shared L3, meaning that it could happen that some cores would push each other out, as despite the output buffers, caches hold frequently accessed instruction, such as some background systems or program related instructions (Hennessy & Patterson, 2019). Re-doing the same calculation for AMAT as in table 8 using a 1 MB L2, would increase it from about 11.9 ns to 25.7 ns (2.4 + 0.97 x (7.6 + 0.375 x (33 + 0.10 x 107))), roughly doubling the memory access time. Since each core would need to wait more for data during filtering, leading to less events being processed per second, reducing the gain from adding cores, they were not pursued further.

Additionally, during the early stages of research, a GPU (graphics processing unit) was considered for boosting processing power, as the filtering program has the structure GPUs are built for, which is the same comparison repeated over many readings. GPUs are designed for this as they contain thousands of simpler cores that perform the same operation on many bytes at once (Hennessy & Patterson, 2019). However, the only calculation done by the program per byte is a checksum and one comparison per reading. The reference detector (Belle II) sends it data into HLT, a computer farm (Itoh et al., 2020) which processes data from all sub-detectors (like the PXD), another detector setup, LHCb (at CERN) move part of their triggers to GPUs because it reconstructs the tracks of particles for every event (Aaij et al., 2020), meaning that much more calculation is done per event than in the filtering program used by PrecisionBeam. In this case, directing data to GPU would take longer than the time the GPU saves on processing (Gregg & Hazelwood, 2011), which would make the speedup unnoticeable. The current processor itself is also designed to perform this kind of simple, repeated work through its SIMD units (single instruction, multiple data) which apply one instruction to several values at once, which is a smaller-scale version of a GPU's approach to processing (Hennessy & Patterson, 2019; Intel Corporation, n.d.-c). Additionally, the 28 cores already provide the required parallelism, at about twelve times the current throughput (table 7). Cost is another reason against a GPU. In today's world the demand for GPU's has grown so much due to AI data centers with no reduction expected in the nearest time, where even an entry-level professional GPU, such as the RTX 4000 Ada, costs about $1,400, almost as much as the recommended processor (Fusion Worldwide, 2026), where a basic CPU would still be required. Including a GPU would therefore create additional spending and power, without a clear reason and a proven benefit, thus it was excluded from the processing stack. If PrecisionBeam changes their filtering program which will include heavier duty calculation for each data, such as track reconstruction, the board's free extension slots (section 4.4) would still allow a GPU to be added without replacing other components.

4.2 Memory

Section 3 identified, that despite the memory not being the root cause, it still isn't enough to support the current requirements. The first is the capacity, where the 4 GB of RAM, limited to 3 GB per process because of the 32-bit OS, can only hold 1.5-4.5 seconds of the incoming data (see table 3), below the required 5-10 s buffer and overhead (justified in section 3). Then, currently the memory path, limited to 6.4 GB/s by the number of RAM channels and their speed (table 3), of which 72% is currently used by the 4.6 GB/s of memory traffic at most (see table 3, RAM row). This 4.6 GB/s comes from the data that is written and read from memory every second. Initially the DAQ writes the detector data (up to 2 GB/s as in table 2) then the processor reads it (up to 2 GB/s as in table 2), then the filtered data needs to be written (up to 0.3 GB/s as in table 2) and then the written data is read out again to be sent on the network, or into the disk (up to 0.3 GB/s as in table 2). Thus 2.0 + 2.0 + 0.3 + 0.3 = 4.6 [GB/s], at most is required, as even though memory handles these requests in turn, all of them must be completed every second. 4.6 GB/s is 72% of the 6.4 GB/s peak, leaving little headroom for bursts. This isn't a bottleneck with the current requirements, however, accounting for future expansion, which is one of the requirements (table 1), it becomes a major bottleneck.

The incoming data rate used to estimate the RAM requirements in section 3 doesn't include the growth in data rate in the future, which the processor was sized for (table 5), as part of the client's requirements (table 1). Therefore, the estimated requirements for the RAM are also re-calculated (see table 10), using the 4 GB/s (table 5) which accounts for future improvements, raising the previous requirement from 12–30 GB (table 4) to 52 GB maximum (Table 10). The bandwidth requirement raises in the same way, since doubling the incoming rate doubles every part of the memory traffic, thus, it becomes about 2 x 4.6 GB/s = 9.2 GB/s at most.

The incoming data rate used to estimate the RAM capacity requirements in section 3 doesn't include the growth in data rate in the future, which the processor was sized for (table 5x), as part of the client's requirements (table 1) Therefore, the estimated requirements for the RAM are also re-calculated (see table 10), using the 4 GB/s (table 5/6?) which accounts for future improvements, raising the previous requirement from 12–30 GB (table 4) to 52 GB maximum (Table 10). The bandwidth requirement rises in the same way, since doubling the incoming rate doubles every part of the memory traffic calculated in section 3, thus, it becomes about 2 x 4.6 GB/s = 9.2 GB/s at most.

Table 11

Memory configurations considered

Modules	Capacity	Bandwidth (heavily depends on the CPU's memory controller as mentioned in table 6) (Kingston Technology, n.d.-a)	Current need (12-30)	Future need (15-52)	Characteristics
1 x 16 GB	16 GB	38.4 GB/s	No	No	Cheapest, but too small even for current buffer need. One channel, and no memory left if the module fails.
2 x 16 GB	32 GB	76.8 GB/s	Borderline	No	Covers the current maximum with almost no margin but it isn't enough to fulfill the future upgrade requirement. It covers 2 channels, so if one module fails there's still the other one, however, the remaining 16 GB cannot hold even a 5 s buffer at the current maximum rate, thus the system would likely have to reduce intensity again.
8 x 8 GB	64 GB	307.2 GB/s	Yes	Yes	Covers capacity needs for future and current requirement but since it uses all eight channels, the bandwidth is the highest bandwidth (8 x 38.4 GB/s). It is far more bandwidth than needed, and if one module fails, the future requirement is still met. However, it leaves every slot filled, thus if further expansion was anticipated, it would require replacing modules rather than adding them, resulting in more cost, and would cause more power to be

					drawn due to all channels being occupied.
4 x 16 GB	64 GB	153.6 GB/s	Yes	Yes, with 12 GB spare	Covers the future capacity requirement with four channels working in parallel. Even if 2 modules fail, full support for current required capacity is still provided. Half of the slots still left for expansion.
2 x 32 GB	64 GB	76.8 GB/s	Yes	Yes, with 12 GB spare	Same capacity, but only two channels. If a module fails, only 32 GB is left. More slots free for expansion

Bandwidth assumes DDR5-4800, at 38.4 GB/s per channel (due to the CPU choice), with one module per channel. Configurations with an odd number of modules, like 3 x 16 GB, 6 x 8 GB or 1 x 32 GB, were not considered, as both fall short of the 52 GB future requirement (48 GB and 32 GB) and channels would be populated unevenly, disabling the memory controller from spreading data evenly across them, reducing the benefit of using several channels. Server motherboards (see section 4.4), therefore specify balanced memory configurations in their population guides (Super Micro Computer, n.d.-c), which was selected based on the CPU (see section 4.1).

A single module provides 38.4 GB/s bandwidth, more than eight times the estimated required for the current detector (4.6 GB/s) and four times the estimated future requirement, 9.2 GB/s, therefore the configuration was chosen by capacity. Of the configurations considered, only 4 x 16 GB and 2 x 32 GB ensure the future improvement requirement is covered (table1), with additional spare capacity. Configurations using 32 GB modules are much less attractive by cost, since at the prices found, 32 GB modules cost about twice as much per GB as 16 GB modules ($24.80 against $11.50; OEMPCWorld, n.d.-a, n.d.-b). Furthermore, with only two of the eight channels in use in the 2 x 32 GB configuration, the memory controller will be capable of serving fewer requests in parallel (requests from cores, DAQ and so on) (Drepper, 2007). The system would also be less resilient, as if one module failed, it would leave 32 GB, covering only the current requirement, but not the full requirement, whereas four 16 GB modules would still leave 48 GB, almost enough to provide for the full requirement. However, using the 32 GB modules leaves more slots for expansion.

As downtime wastes expensive beam time, and the client's budget is limited, four 16 GB modules were chosen, still leaving four slots free for further expansion. The modules are also interchangeable, meaning the facility could buy two now, covering the current requirement (32 GB against 30 GB), and add the other two when the detector upgrade is anticipated, or when more financing arrives, spreading the cost. However, as two

modules only just meet the current requirement, buying all four at once is still the recommended option.

The proposed memory is therefore 64 GB, made up of 4 x 16 GB DDR5-4800 registered ECC modules (Samsung M321R2GA3BB6-CQK). Its specifications are compared with the current memory in Table 12.

Table 12

Comparison of current and recommended memory

Specification	Current memory (Serverblink, n.d.)	Recommended memory (Exxact, n.d.)	Context
Type	DDR2-400 registered ECC	DDR5-4800 registered ECC	DDR5 is three generations newer, affecting its possible characteristics like module data transfer rate, which is twelve times faster in this particular model (4,800 against 400 MT/s). It also uses less power for the same work, as it runs at a lower voltage (1.1 V against 1.8 V) (Exxact, n.d.; Serverblink, n.d.). The generation also allows the capacity per module to be much higher (16 GB against 1 GB), reaching the required capacity with only four of the eight slots. Both are registered, meaning they reduce the load on the memory controller and therefore support higher loads (Mellor, 2022), where both also use ECC to correct memory errors (further explanation below). Its

			choice also depends on the CPU largely, as it supports DDR5-4800, the memory should be the same, as it can only run as fast as the CPU (memory controller inside processor) allows (see table 6).
Modules and capacity	4 x 1 GB Total of 4GB	4 x 16 GB Total of 64 GB	Covers the current and future requirements (Table 10).
Channels	2 of 2	4 of 8	Governed by the processor for the new RAM (Intel Corporation, n.d.-c), with the motherboard having one slot per channel (Super Micro Computer, n.d.-c), thus it affects how much can be transferred simultaneously (increasing bandwidth).
Speed and bandwidth	400 MT/s → 3.2 GB/s (as each transfer moves 8 bytes 400 x 8 = 3200 MB/s) 2 x 3.2 = **6.4 GB/s**	4,800 MT/s → 38.4 GB/s (as each transfer moves 8 bytes 4,800 x 8 = 38,400 MB/s) 4 x 38.4 = **153.6 GB/s**	About 24 times the current bandwidth, and about 17 times the 9.2 GB/s needed at the future rate – well above the requirement. Additionally, a RAM with lower data transfer rate could have been considered, but since this is the lowest speed sold of DDR5 (Kingston Technology, n.d.-b), the choice was clear.
CAS latency (true latency)	CL3 (3 x 2,000 ÷ 400 = 15.0 ns)	CL40 (40 x 2,000 ÷ 4,800 ≈ 16.7 ns)	About the same. CAS latency (CL) is the number of memory clock cycles that pass between when memory receives a request of data, until RAM forms a response and is ready to forward it on the bus. Since DDR (double-data rate) means the memory transfers data twice per clock cycle, its clock runs at half its

			data rate, so the true latency in nanoseconds is CL x 2,000 ÷ data rate. The new memory has more than ten times as many cycles, but each cycle is twelve times shorter (2,400 ÷ 200 = 12), thus, the actual waiting time barely changes. Suited to this scenario, where bandwidth and capacity matter more than anything. The filtering program reads each event in order, from start to end, therefore, the processor can predict which data comes next and load it ahead of time (Drepper, 2007), thus, the wait for RAM to compose a response, overlaps in time with the work being done by CPU on the data already loaded. The only limit to the program is how much data memory can deliver per second, not how long the first piece takes to arrive, thus, latency was not a big factor during making the choice.
Price	-	About $183.85 per module (OEMPCWorld, n.d.-a) (price differs depending on the vendor)	-

ECC corrects single-bit memory errors and detects double-bit memory errors by storing extra check bits with each block of data, allowing for easy detection and fix (Hennessy &

Patterson, 2019). Therefore, the filtered events are checked for corruption while they wait in the memory to be sent out or saved to storage. Registered means that there is a small buffer, between the memory controller and chips which store data inside RAM, which stores the control and address signals, but not the data itself, before they reach the chips. This reduces the electrical load on the memory controller, and makes the modules more reliable, compared to unregistered modules, which is why registered memory is standard in servers (Mellor, 2022). Since the system runs continuously, handles valuable data, and has many RAM modules, registered ECC support was kept.

Furthermore, the Samsung M321R2GA3BB6-CQK was chosen as it matches all of the requirements without paying for unused features. It is also certified for Supermicro systems (Hyperscalers, n.d.), which is the same platform as the recommended motherboard (see section 4.4). Its speed matches the processor's supported maximum, 16 GB capacity allows the balanced four-module configuration chosen above, and its ECC protects the data it holds, making it the chosen recommended RAM.

DMA controller (direct memory access) allows components like storage, DAQ card or network cards to access memory without involving the processor's cores in the process (Tanenbaum & Bos, 2015). In modern system, each component has its own DMA. The proposed system exploits this feature, as the DAQ card writes incoming events into memory while the 28 cores are filtering earlier events, so data arrives and is processed at the same time. The DAQ card writes new events into free buffer space while the cores read earlier ones in turns, managed by the memory controller, thus the two do not compete for the same data.

As mentioned in section 4.1, the first step of a CPU fetching data is resolving its virtual memory address. It originates from a concept called virtual memory where the OS gives each program the illusion of exclusive access to the memory. Each program uses virtual memory addresses, which the CPU maps to physical addresses, through the use of TLB, a small cache dedicated to storing most recent translations, which stores the most recently accessed translations, and a page table stored in RAM, which stores all translations together with markings to which data is not in the primary memory. If data, currently not in primary memory, is requested, a page fault occurs, which means that data is loaded from storage, moving other data out if needed. Fetching data from storage usually takes thousands of times longer, thus, as mentioned in section 3, high rate of page faults is undesired, happening constantly is called thrashing, suggesting a shortage of RAM (Tanenbaum & Bos, 2015). With 64 GB, above even the 52 GB future requirement, all of the system's working data is expected to stay in RAM, minimizing

page faults, during normal operation. The 64-bit OS (explored in section 4.4) also removes the 4 GB limit per process (see section 3), which confirms there will be no restriction of keeping all working data in the RAM.

Overall, the recommended memory removes the limitations identified in section 3. At about $735, it was sized to the requirement, keeping the cost of currently expensive DDR5 memory to what the system needs. Furthermore, this option allows the facility to split the budget, considering its tightness, with still providing full support to the new system, and achieving their requirement of accounting for future expansion in the long term. This isn't recommended as the same RAM modules may be harder to find later, possibly at a higher price as RAM prices are currently rising (Fusion Worldwide, 2026). Additionally, it would require shutting the server down in order to add the modules. Therefore, since it fits in the budget, buying all modules now is recommended.

4.3 Storage

The current system provides 3 TB usable storage, far below the 8.64–25.92 TB needed to hold a three-day backlog and writes at most at about 315 MB/s, barely sufficing the 300 MB/s maximum output requirement (section 3), at future rate these requirements increase to 51.84 TB and 600 MB/s (Appendix B.6). Both event and system storage are replaced, while maintain the storage separation, where each is allocated different kinds of drives.

The replacement decision is further supported by the drives and RAID card dating from 2008, with the card using PCI-X, which modern motherboards (required for the CPU (section 4.1)) would no longer support. Nevertheless, the current drives could physically connect to the new system, as the motherboard's SATA ports accept older SATA II drives (section 4.4), but since they add only 3 TB, occupy 4 slots, are desktop drives (not designed for continuous operation) and after about 18 years they are far beyond their expected service life (Seagate Technology LLC, 2007a), it is considered unbeneficial. Therefore, the decision is to replace the event storage fully.

The OS drives are also desktop hard drives from the same period. Thus, like the event storage, they're not designed for continuous operation and well past their expected service life (Seagate Technology LLC, 2007b), making a failure become far more likely. They could also connect to the SATA ports, but HDDs are not recommended for the kind

of reads which system related tasks perform (explore further below) and their read speed of 106 MB/s at most (table 3) would keep the booting and program loading slow. Additionally, re-using the old drives, would mean less SATA ports available for the event storage and leaving the new board's M.2 slots unused (section 4.4). Therefore, they are also replaced fully.

Storage is the slowest type of memory but cheaper and persistent (Hennessy & Patterson, 2019) compared to others, which suits an outage buffer, that stores filtered event data before its sent to the analysis facility. Since, the filtering program is assumed to write in large sequential blocks (section 4.1), SSDs' fast random access provides nothing, unlike for OS and programs, the system accesses many different locations when booting, loading the filtering program and running background tasks. Thus, for the outage buffer, hard disk drives (HDDs) were chosen over solid-state drives (SSDs) as they also cost considerably less (about $25-$60 per TB against about $75-$100; (ORM Systems, 2026; Amazon Web Services, n.d.), a significant difference within the client's budget (see table 1). The HDD array can be selected to exceed the required write speed (eexplored later), hence there's no incentive for an SSD. Unlike, for the OS and program files, where this fast random access is beneficial, as the system reads accesses many different locations when booting, loading the filtering program and running background tasks. Thus the same separation of system and event storage will be kept, allowing the event storage array to be repaired or replaced without affecting the operating system.

Thus, two Kingston DC2000B 240 GB enterprise M.2 NVMe SSDs (part number SEDC2000BM8/240G) are used, at about $310 each (eBay, n.d.) for system storage. M.2 is the format that mounts directly on the motherboard and NVMe is a protocol/interface allowing the drive to use the faster PCIe bus instead of SATA (unlike HDDs) for faster access (Kingston Technology, n.d.-c), leaving the SATA ports for the event storage drives. The operating system chosen (Section 4.4) requires at least 10 GiB (about 10.7 GB) of disk space to install (Red Hat, n.d.-b), so the smallest possible enterprise designed drive is chosen; 240 GB which is about 22 times the minimum, leaving plenty of room for the filtering program, system logs and updates, without unnecessary spending. As specified by the producer, the drive is designed for continuous server workloads, with hardware-based power loss protection, a rated lifetime of 2 million hours between failures (which suggests the mean time before the next failure based on a probability rating calculated from the number of failures observed in a group, meaning that in reality if a pool of 2000 drives was used, a failure could be observed every 1000 hours of their work) and a five-year warranty (Kingston Technology, n.d.-c). It also reads at up to 4,500 MB/s and writes at up to 400 MB/s (Kingston Technology, n.d.-c), and when compared with read speed of about 106 MB/s for the current OS drives (table 3), booting

and loading the program would occur about 40 times faster, while 400 MB/s writes exceeds the current event drives' speed four times, which sustains what the systen requires and speeds up the updating and log creation process. Two are required as the previous RAID 1 mirroring will be kept, thus the system keeps running and expensive beam time (section 2) is not wasted if one drive fails.

RAID is a technology which combines several drives into one logical unit, where different levels provide different mechanisms like, doing mirroring as a backup practice or storing parity (taking up some of the total space) to reconstruct data if a drive fails (Patterson et al.,1988). RAID 1 is the simplest and safest way to achieve this with only 2 drives (as RAID 5 or 6 require at least three or four) (Layton, 2026). For the event storage drives, the current system uses RAID 5, which takes up one drive for parity storage and survives one drive failure. However, RAID 5 becomes risky with a failed drive in large modern drives, because the rebuild must read every remaining drive in full, and since the larger the drives, the higher the probability of an unrecoverable read error occuring, as there are more bits to go thorugh, a single such error can stop the rebuild and lose data (GPUGuru, 2026). RAID 6 stores two sets of parity, thus it survives two drive failures, or one failure and a read error occurred during the rebuild, therefore it's recommended for large storages (Need to Know IT, 2026; Layton, 2026). Since it stores 2 sets of parity, it would need extra storage, and since usable capacity is (N − 2) x drive size, so holding the 51.84 TB future backlog opens a variety of options (see table 14), where the best one would be choosing the 16 TB version. RAID 6, against RAID 5 would cost one more drive, so about $625 (estimated from table 14) . Since the event storage is expected to be usually empty (section 3), however during a possible outage it holds all of the filtered data which cannot be recreated, and losing this data would cause a waste of the expensive beam time. Based on this the extra drive is a small cost. Other levels were also considered, with RAID 0 excluded as it provides no redundancy (which is too risky in this case), RAID 10 has recovery speed benefits, however since it mirrors each drive onto a second one it uses half of the total capacity for mirroring (Layton, 2026), thus 8 drives would be needed in total costing about $1250 more. The speed of RAID 10's could be beneficial, but considering the budget constraints, RAID 6 was chosen, as it offers the required protection at a lower cost.

Table 14

Event storage configurations

Configuration	Usable capacity	Bays used	Power in use (approx.)	Total cost (approx.)	Analysis

9 x 8 TB	(9 − 2) x 8 = 56 TB	9	about 100 W	9 x $600 (Newegg, n.d.-a) = $5400	The cheapest drives individually, but more is needed, also the highest power draw and the most drives that can fail, though each rebuilds quickest. Also uses 9 bays, which is increases the requirements for the motherboard, the most of every option.
7 x 12 TB	(7 − 2) x 12 = 60 TB	7	About 70 W	7 x $645 (Newegg, n.d.-b) = $4515	Comes out cheaper than first option, less power and less bays used, providing more actual usable capacity and less drives that can fail
6 x 16 TB (chosen)	(6 − 2) x 16 = 64 TB	6	About 60 W	6 x $625 (Newegg, n.d.-c) = $3750	The cheapest option with the most capacity. Not the lowest power, but second best option. Occupies less bays, making it second lowest.
5 x 20 TB	(5 − 2) x 20 = 60 TB	5	About 50 W	5 x $882 (Newegg, n.d.-d) = $4410	Fewest drives, lowest power, and second cheapest option, but takes the longest to rebuild.

The sizes of the drives used must be the same as RAID treats all drives as same size. 16 TB option seems significantly cheaper, yet well balanced.

Furthermore, for the event storage, the recommended drive is the Seagate Exos X16 16 TB (part number ST16000NM001G), at about $625 per drive new (Newegg, n.d.-c). Six of these would be needed to sustain capacity needs (see table 13) and RAID requirements. As an enterprise drive, it is designed for continuous operation in servers, with a rated lifetime of 2.5 million hours between failures, with each drive also writing up to 261 MB/s (Seagate Technology LLC, 2019), thus with four drives worth of data, it would be up to 1,044 MB/s when writing large blocks, as the filtering program does in 1.5 MiB blocks (table 6), well above the 600 MB/s future requirement (table 13).

Additionally, the current system uses a hardware RAID card to manage the RAID functionality, whereas in the new system, no card is used, for reasons seen in table 15. The final comparison of the current and recommended storage are in table 16.

Table 15

Reasons behind software RAID rather than hardware

Reason	Explanation
Lack of support	Existing card uses PCI-X, which is no longer supported (see table 3)
Hardware is no longer needed	The drives can connect directly to the motherboard with enough SATA ports (see section 4.4)
Parity calculation cost	Software would have to use the system's capacity for the parity calculations, however extra 28-cores, chosen in section 4.1, ensure plenty of spare capacity
No additional configuration	The RAID software is already included in the Linux operating system chosen (see section 4.4)

Table 16

Comparison of the current and recommended storage

Specification	Current	Recommended	Context
OS drives	2 x 250 GB SATA II HDD, RAID 1 (Seagate Technology LLC, 2007b)	2 x 240 GB Kingston DC2000B M.2 NVMe SSD, RAID 1 (Kingston Technology, n.d.-c)	M.2 is the format that mounts directly on the motherboard and NVMe is a protocol/interface allowing the drive to use the faster PCIe bus instead of SATA (unlike HDDs) for faster access (Kingston Technology, n.d.-c), leaving the SATA ports for the event storage drives. The operating system chosen (section 4.4) requires at least 10 GiB (about 10.7 GB) of disk space to install (Red Hat, n.d.-b), so the smallest possible enterprise designed drive is chosen; 240 GB which is about 22 times the minimum, leaving plenty of room for the

			filtering program, system logs and updates, without unnecessary spending. As specified by the producer, the drive is designed for continuous server workloads, with hardware-based power loss protection, a rated lifetime of 2 million hours between failures (which suggests the mean time before the next failure based on a probability rating calculated from the number of failures observed in a group, meaning that in reality if a pool of 2000 drives was used, a failure could be observed every 1000 hours of their work) and a five-year warranty (Kingston Technology, n.d.-c).
RAID level of OS drives	RAID 1	RAID 1	Mirroring across a second drive, with the small capacity, it is the safest choice.
Speed of OS drives	Read of about 106 MB/s (table 3)	reads at up to 4,500 MB/s and writes at up to 400 MB/s (Kingston Technology, n.d.-c)	When recommended is compared against the current booting and loading the program would occur about 40 times faster, while 400 MB/s writes exceeds the current event drives' speed four times, which sustains what the systen requires and speeds up the updating and log creation process.
Event drives	4 x 1 TB Seagate Barracuda 7200.11 (Seagate Technology LLC, 2007a)	6 x 16 TB Seagate Exos X16 (Seagate Technology LLC, 2019)	Recommended drives are enterprise drives made for 24/7 operation with low failure occurences
RAID level of event drives	RAID 5 (one drive's parity)	RAID 6 (two drives' parity)	Recommended survives two failures, or one failure plus a read error during rebuild which can happen more frequently with the curently bigger storage size.
Usable capacity of event drives	3 TB	64 TB	Account for both the current backlog 25.92 TB; and the future 51.84 TB (Table 13)
Write speed of event drives (large blocks)	About 315 MB/s	About 1,044 MB/s	Recommended covers for future requirement of 600 MB/s

RAID controller	3ware 9550SXU card, PCI-X	Linux software RAID (mdadm)	No card or slot needed, removing extra cost and releasing occupation space from the server.
Interface	SATA II (3 Gb/s)	SATA III (6 Gb/s) for data; NVMe (PCIe 4.0) for OS	Each SATA III port (about 600 MB/s) is over twice as fast as one drive, ensuring each drive imposes the limit of data transfer rate.
Power	About 45 W for event storage and 16 W for system storage (table 3)	For event storage up to 60 W in use, 30 W idle (6 x 10 W and 6 x 5 W) (Seagate Technology LLC, 2019), and system storage about 8 W (Kingston Technology, n.d.-c; 4.02 W average write)	Total of about 68 W against the previous 61 W, small increase, still supported by the UPS (table 3)
Price	-	About $3,750 for data drives (Newegg, n.d.-c) and $620 for OS drives (eBay, n.d.)	About $4370

The recommended event storage provides enough capacity to support a 3-day backlog in case of a network downtime, has about twice as much write speed as needed for the future requirements (about 1,044 MB/s against 600 MB/s), with protection against two drive failures instead of one (through RAID 6), made in accordance with the much bigger capacity. The separate, mirrored (through RAID 1) SSDs keep the operating system running independently of the event storage. This storage configuration requires six SATA connections and drive bays for the data drives, and two M.2 slots for the system drives with PCIe ports, which the motherboard and chassis must provide (sections 4.4 and 4.5). Additional error detection and maintenance like health monitoring and hot-swap drive replacement, are described in section 4.8.

The purchase, like in the case of RAM, could be divided into 2 to spread the cost. Since RAID 6 requires four drives at least, it could be the initial purchase giving (4 − 2) x 16 = 32 TB, which sustains the current three-day backlog of 25.92 TB (see table 13). The remaining two could be added later, or when a detector upgrade is anticipated keeping the same level of protection. However, increasing the number of drives could create up to days of overhead, as the software needs to spread existing data across the new drives. One purchase is therefore recommended, but the staged option is a possible reasonable alternative if the budget required spreading the cost.

4.4 Motherboard, OS, Network, Graphics and I/O

Current motherboard, OS and network connector aren't suitable for re-use. The current motherboard supports only single-core Nocona processors, DDR2 memory and PCI-X expansion cards, with one first-generation PCIe slot (table 3). Meaning that both the chosen processor (section 4.1) and memory (section 4.2) will not be compatible. Additionally, it contains only two SATA connections and no M.2 slots (Super Micro Computer, 2007), meaning it wouldn't be able to support the storage choices, this is not because the storage is demanding, but because it predates the current, commonly used M.2 slots, which did not exist when the board was made. Current network controller is built into the board, thus will be replaced with it. Nevertheless, the current network controller barely sustained the minimum network rate requirement (section 3 and table 3), thus it would need to be replaced either way. The current OS limits each process to only about 3 GB of memory (section 3), which is what the filtering program and DAQ would be limited to, and because the OS is also discontinued (Fermilab & CERN, 2022), replacement is recommended; a 64-bit system which allows for up to around 256 TB, of virtual address space (Intel Corporation, 2023), far beyond the size of the recommended RAM.

Additionally, the current motherboard limits the current system in a variety of ways. First, the DAQ card is connected through a PCIe slot of an old generation which limits the DAQ card to about 2 GB/s, leaving no headroom for current requirements, and full lack of support for the future. The, the built-in network card supports a data transfer rate at 1 GbE, that only barely sustains the current minimum transfer rate (table 2). Also, the memory controller sits in the motherboard, instead of the CPU, thus all traffic from the processor needs to pass over the front-side bus, limited to 6.4 GB/s.

The recommended OS is therefore AlmaLinux 9, a free, 64-bit rebuild of Red Hat Enterprise Linux (RHEL) 9 and the distribution recommended by CERN and Fermilab for physics computing (Fermilab & CERN, 2022). RHEL itself requires a paid subscription, while AlmaLinux shares its hardware support and documentation (Red Hat, n.d.-b), and includes the mdadm software RAID used for storage (section 4.3), without the extra cost, which the budget favors. Since the filtering program is already available on the 64-bit version (see section 3), it would just require re-installing, which would be done together with the system configuration, and it would allow the program to use the full 64 GB of memory (section 4.2). The DAQ card is assumed to be compatible as the recommended OS is from the same family as the current system, which should be confirmed with the card manufacturer.

The previous sections set the requirements for the motherboard: an LGA4677 socket supporting 5th-generation Xeon processors (section 4.1), at least four DDR5 memory channels and at least 4 slots (section 4.2), a PCIe slot for the reused DAQ card (section 4.1) (PCIe 2.0 x8 or more to provide the minimum bandwidth required (4 GB/s; Nandra, 2009) for future expansion (table 5)), six SATA connections and two M.2 slots with PCIe connections for the storage (section 4.3). Additionally, the future output rate of 600 MB/s (table 13) sets the required network speed, which could be built into the board, or be a separate network card. Since Ethernet in practice delivers at most 94% of its rated speed (IBM, 2026), the current 1 GbE is therefore about 118 MB/s, and the next one fitting the new requirement is 5 GbE so about 590 MB/s, and realistically even less would be achieved, which is just under the requirement. The next possible is 10 GbE, which will be the aim when choosing a network connector. As the processor is a single-socket model, only single-socket boards were considered. Many brands had boards supporting some requirements, Supermicro had 3 fitting options which were analyzed (see table 17). The reason behind choosing Supermicro was mainly because it is optimized for mainstream server applications (Super Micro Computer, 2024). They also sell products as separate, compatible components rather than as complete servers (Super Micro Computer, n.d.-f), and the current server is also built on Supermicro hardware, where the recommended memory is certified for it (section 4.2), thus ensuring support across the components, and reducing adjustment time for the staff.

Table 17

Comparison of motherboards considered

Motherboard	X13SEI-TF (chosen) (Super	X13SEI-F (Super Micro Computer, n.d.-d)	X13SEW-TF (Super Micro Computer, n.d.-e)

	Micro Computer, n.d.-c)		
Network	2 x 10 GbE (Intel X550)	2 x 1 GbE (Intel I210)	2 x 10 GbE (Intel X550)
Expansion slots	2 x PCIe 5.0 x16, 3 x PCIe 5.0 x8	2 x PCIe 5.0 x16, 3 x PCIe 5.0 x8	1 x PCIe 5.0 x8 (in an x16 slot), 1 x PCIe 5.0 x32
Storage connections	10 x SATA, 2 x M.2	10 x SATA, 2 x M.2	10 x NVMe via MCIO connectors, 10 x SATA, 1 x M.2 (PCIe 3.0 x2)
Price	About $891 (Newegg, n.d.-e)	About $691 (Newegg, n.d.-f)	About $800 (Newegg, n.d.-g)
Fit for PrecisionBeam	Meets every requirement but is also the most expensive	Would require a separate 10 GbE card (ranging from $50 up to $300 depending on whether new or used (eBay, n.d.-b; Getprice.pl, n.d.)	Built for many NVMe drives (SSD's), which are not used, with fewer ordinary slots for future cards, like another DAQ
Conclusion	Most expensive but provides all requirements.	Mostly the same as first option but would require an additional NIC, which cost depends on whether it's new or used. Used cards don't have a warranty and aren't the best option as information about their condition is usually limited, and because the facility would send up to around 18 TB of data daily (table 13), they would rely heavily on the network connector. However, even the cheapest cards reduce the cost by at most $150, which is less than 1.5% of the budget. The card would also occupy an extra slot.	Less expensive than the other but lacks one M.2 for the second mirrored OS drive, thus an adapter would be needed to connect it through the MCIO, which could eat at the saving. Also has 3 less expansion slots than the others, thus if it is decided that a GPU and another DAQ card is added in the future, it may become an issue.
	All boards have eight memory slots, one per channel, supporting DDR5 up to 5600 MT/s (Super Micro Computer, n.d.-c, n.d.-d, n.d.-e). As the processor limits memory to DDR5-4800 (section 4.1), each gives the same maximum memory bandwidth, thus memory does not distinguish between them. All boards also provide more than required generation of the expansion slot connection for the DAQ (explained in footnote), with the third		

	one providing one less slot. First and third boards provide twice the required network connector specification. The third option also adds complexity due to lack of an M.2 port, which could eat at the $91 saved, since it's a very small saving, it was discarded. First and second choice were considered, but since the network connection is a single point of failure for sending data to the analysis facility, relying on a used controller with unknown history instead of a new, warrantied onboard controller, at a maximum saving of $150, doesn't appear worthy. Since the extra spending fits within the budget, the recommendation is therefore the X13SEI-TF.

Purple suggests specifications which do not cover the requirements for the system. DAQ, the PCIe 3.0 x8 card (table 3) can be installed in one of the PCIe 5.0 x16 slots because PCIe is backwards compatible, where the limit of 8 GB/s of bandwidth will be imposed by the DAQ card (table 3), which is twice the bandwidth required, accounted for future expansion (4 GB/s as in table 5). The remaining expansion slots are beneficial, as they could be used for future cards (another DAQ), a GPU if the filtering ever requires one (as explored in 4.1), meeting the client's requirement for future expansion (table 1).

The board's two 10 GbE ports each carry 10 Gb/s (10 ÷ 8 = 1.25 GB/s), thus about 1.175 GB/s at the 94% achieved in practice (IBM, 2026), almost twice the 600 MB/s needed (table 13). The spare capacity matters after an outage, where the storage should be emptied out after the network is fixed, and the more the spare capacity there is, the faster this will occur. In the recommended system, only one port carries the filtered data causing a 3-day backlog to clear out in 2.2-25h (see table 27), while the second acts as a failover. Alternatively, both ports could be used for transmitting data causing the clear out speeds to increase to 1.1-8.2h (see table 27). However, the decision for switching would require an analysis of the facility's switch and its link to the analysis facility to support it, which is outside the scope of this report, but is acknowledged as a requirement and a possible extension.

Table 27

Time to clear a three-day backlog over one 10 GbE port (about 1,175 MB/s in practice) and over two 10 GbE ports (about 2,350 MB/s in practice)

Case	Current minimum	Current maximum	Future maximum
Output rate (MB/s)	100	300	600
3-day backlog (table 13) (TB)	8.64	25.92	51.84
Spare capacity at single port (network capacity in practice – output rate)	1,075 MB/s	875 MB/s	575 MB/s

Time taken at a single port (h)	2.2	8.2	25
Spare capacity at a double port (network capacity in practice – output rate)	2,250 MB/s	2,050 MB/s	1,750 MB/s
Time taken at a double port 9h)	1.1	3.5	8.2

The server runs without a monitor or keyboard, nevertheless the recommended board provides a VGA port, which is driven by the basic 2D graphics built into the board (Super Micro Computer, n.d.-c), which is sufficient for setup and the remote console; no dedicated GPU is needed (Section 4.1). Together with USB 3.2 ports (5 Gb/s = 0.625 GB/s = 625 MB/s) about ten times faster than the current USB 2.0 (480 Mb/s = 60 MB/s) (table 18) allowing for setup and maintenance operations like installing all the software or updating it. There are no printers etc. as the system has no user interface, where the results are analyzed by the analysis facility. The board also includes a baseboard management controller (BMC), a microcontroller (compact computer) embedded on the board (Super Micro Computer, n.d.-c), which lets staff access the server (through SSH, KVM-over-IP, etc.) as it either shares the network port or has its own (Super Micro Computer, 2020). It enables monitoring, updating, console access, restarting and much more (Super Micro Computer, 2020), without a BMC all of those operations would need to be handled in person, creating delay and inconvenience. The board's PCIe devices also signal the processor using message-signaled interrupts rather than the current board's shared interrupt lines (table 18), described with the other connections between components in section 4.7. The recommended components are compared with the current ones in table 18.

Table 18

Comparison of the current and recommended motherboard

Specification	Current (Supermicro X6DHE-XG2) (Super Micro Computer, 2007)	Recommended (Supermicro X13SEI-TF) (Super Micro Computer, n.d.-c)	Context
Socket and chipset	2 x Socket 604, Intel E7520	1 x LGA4677, Intel C741	In recommended the chipset handles only slower devices (SATA, USB), as memory and PCIe connect to the processor

			directly (see section 4.1), removing any overhead from faster components like in the previous system (table 3).
Memory	8 slots, DDR2-400, registered ECC	8 slots, DDR5 up to 5600 MT/s, registered ECC (RDIMM)	Supports the 4 x 16 GB DDR5-4800 chosen in section 4.2, with four slots free, and ensures the same crucial error handling is kept (section 4.3 and 4.8).
Expansion slots	5 x PCI-X, 1 x PCIe 1.0 x8	2 x PCIe 5.0 x16, 3 x PCIe 5.0 x8	The reused DAQ card can run at its full speed with the recommended system with four slots free for future cards.
Storage connections	2 x SATA on the board (about 150 MB/s each), plus the RAID card's SATA II ports (about 300 MB/s each) in its PCI-X slot (about 1.06 GB/s shared) (table 3)	10 x SATA III (about 600 MB/s each), 2 x M.2 (NVMe, PCIe 5.0 x4, about 15.8 GB/s each)	Each SATA III port supports up to about 600 MB/s, which is over double of one data drive write bandwidth (261 MB/s) (section 4.3), and each M.2 slot for the system storage, about 15.8 GB/s, which is over three times the SSD drives' 4.5 GB/s. As each drive has its own port, adding drives does not divide this speed between them like in the current system where a RAID card connects 4 drives, which share the PCI-X bus (no longer supported and needed, see section 4.3). Thus, the connections leave room for faster drives in the future without imposing limits.
Network	2 x 1 GbE (about 118 MB/s in practice)	2 x 10 GbE (about 1,175 MB/s each in practice), plus 1 x dedicated management port (for BMC)	Supports the maximum required network speeds, and speeds up clearing backlog. Additionally, the network port for BMC ensures remote management (explored below), which traffic is separated from the event data sent to the analysis facility thanks to the separate port.

Remote management	None installed	ASPEED AST2600 BMC (IPMI)	A BMC has own processor, firmware and network connection, as mentioned previously, which runs independently of the main system, and stays powered whenever the server is switched off, but it needs to be plugged. It continuously monitors the server's sensors (temperatures, fan speeds, voltages) and keeps a log of hardware events. It provides the staff with many possibilities regarding the server's maintenance, monitoring and control from a remote location, providing convenience and efficiency. The possible tasks include switching the server on, off or restart it, view and control its screen and keyboard remotely, perform installations and updates, even to the BIOS and firmware (Super Micro Computer, 2020). Since it works even when the system is turned off or not working for another reason, it allows for diagnosing and fixing faults remotely, which the current server, lacking a BMC, cannot do. This is further addressed in section 4.8.
Rear ports	1 x VGA, 2 x USB 2.0 (60 MB/s), PS/2, serial	1 x VGA, 2 x USB 3.2 (625 MB/s), 2 x USB 2.0 (60 MB/s)	Used for set-up and maintenance, largely the same, with 2 much faster USB ports allowing for quicker downloads or installations.
Graphics	Onboard 2D video only	Basic 2D video built into the	Sufficient for setup and remote console access; no GPU was established as needed for the

		BMC (ASPEED AST2600)	processing (filtering program running on multiple cores) (section 4.1)
Interrupts	Shared physical interrupt lines, managed by the chipset's interrupt controllers	Supports message-signaled interrupts (MSI-X) for PCIe devices, including the DAQ card and the 10 GbE controller (Nguyen et al., n.d.)	An interrupt is a signal sent by a component of a system to the processor to notify it of a completed task, a change or anything that needs the processor's attention (Tanenbaum & Bos, 2015). For example, when the DAQ card has written new events. Interrupt handlers are designed to be short, as to not pause the system's operations for a noticeable time. The current board connects several components over a shared set of physical wires, thus the processor must verify which device signaled, where busy devices can delay each other (Nguyen et al., n.d.). With MSI-X, each device has its own separate signals, that are written to memory, where the OS can spread the signals across cores, rather than have one do all the work as in the current system (Nguyen et al., n.d.), keeping interrupt handling within the processor capacity reserved for system tasks (table 5).
Price	-	About $891 (Newegg, n.d.-e)	

On the X13SEI-TF, RAM, DAQ card (in a PCIe 5.0 x16 slot) and the two SSDs (in the M.2 NVMe PCIE 5.0 slots) connect directly to the processor, while the six HDDs connect to

the SATA ports of the board's Intel C741 chipset. The chipset itself has a built-in RAID support for levels 0, 1, 5 and 10 but not RAID 6 (Super Micro Computer, n.d.-c), confirming the choice of software RAID (section 4.3), as the RAID level for HDDs isn't supported, and the SSDs aren't connected to the chipset. How data moves between these components, including DMA and memory arbitration, is described in section 4.7, whereas a summary of the maintenance, error handling and detecting in section 4.8.

Ultimately, the recommended board doesn't limit the bandwidth of any of the components, with processor having direct connection to the memory and PCIe slots (thus DAQ card), and with the event storage through the chipset without a separate card. The network speed is also adjusted to sustain the new requirements, with a second port for redundancy, ensuring that in case of a port failure, the system could still be uninterrupted. Additionally, the 4 free PCIe slots, 4 free memory slots and 4 free SATA ports, there is clear room for future expansion, limited for the drives only by the bays on the chassis (section 4.5).

4.5 Chassis, cooling and PSU

The company reported overheating at full memory and processor utilization, forcing the intensity down (section 3) which was caused by the workload. However, since the recommended processor uses nearly twice as much power as the current one (185 W against 103 W; section 3 and 4.1) with addition of the other components, meaning it needs to dissipate that much heat, the current cooling would not suffice. The current chassis' cooling also cannot be increased as all four of its fan positions are already filled (Super Micro Computer, n.d.-a). Modern Supermicro's 2U heatsinks for this processor, like the passive SNK-P0088P, are rated for up to 350 W (Super Micro Computer, n.d.-c, n.d.-i), however, they rely on the strong airflow of current 2U chassis designs and usually come with stronger fans, which the current chassis, built in 2008 around a 103 W processor, cannot provide and would still require replacing the fans. The current chassis also has only six drive bays, all of which utilized by the new event storage (section 4.3), leaving no room for future expansion, also resulting in limiting the recommended motherboards capabilities, where for example the four remaining SATA ports (section 4.4) could never be used. The 2U height also constraints any components in the expansion slots to low-profile (table 3), where fewer cards are made in this size, causing them to be more expensive or less capable that their full-size versions (Štefan, 2025). Thus, if the facility decided to add a GPU in the future (section 4.1) they would be restricted by this chassis, potentially costing more. Therefore, the current 2U chassis cannot be reused, and 4U chassis is recommended.

The power supply would also need replacing, as the current supply's 400 W rating, leaves no margin for the recommended set-up (section 4.9) where a supply running near its limit is less efficient (Mpitziopoulos, 2018), thus more likely to fail. The supply is also about 18 years old, making it increasingly unreliable and as the components inside power supplies wear with age, and since there is only one it is non-redundant, thus, its failure would stop the whole server, wasting expensive beam time (section 2).

The recommended chassis is therefore the Supermicro SC745BAC-R1K23B, a 4U chassis designed for Supermicro's motherboards that comes with two redundant 1200 W power supplies (Super Micro Computer, n.d.-g). As the chassis does not include a heatsink for the CPU, Supermicro's SNK-P0090AP4 is recommended, designed for 4U chassis and supporting processors up to 350 W, which is almost twice the recommended processor's 185 W, and is also compatible with the X13SEI-TF (Super Micro Computer, n.d.-c). It is also a small expense (table 19), and Supermicro lists the heatsink for this motherboard in 4U chassis (Super Micro Computer, n.d.-c). Together with the chassis' five fans this addresses any possible overheating that was previously present.

Table 19

Comparison of the current and recommended chassis (including PSU and cooling)

Specification	Current (CSE-822T-400LPB) (Newegg, 2008)	Recommended (SC745BAC-R1K23B) (Super Micro Computer, n.d.-g)	Context
Height	2U	4U (has to be rack-mounted with the CSE-PT26L-B rail kit)	The new chassis comes in a format not supporting a rack placement; thus, a rail kit is used. It is also bigger than the previous one, reducing the need for low-profile components.
Cooling	4 x 80 mm fans	5 x 80 mm hot-swap fans, and a Supermicro SNK-P0090AP4 4U heatsink (up to 350	Exceed the requirements, ensuring that future expansion is accounted for and the overheating issue is solved.

		W) (Super Micro Computer, n.d.-h)	
Power supply	1 x 400 W, non-redundant, about 18 years old	2 x 1200 W, redundant, 80 PLUS Titanium	The recommended one includes a highest tier certification that the PSU has, meaning it delivers about 94% of the power drawn at half load, against at least 80% for the current supply, so less power is lost as heat (Mpitziopoulos, 2018), compared to the previous PSU which didn't have this certification, contributing to a lower electricity bill. Despite 1200 W surpassing the UPS provision, it is not an issue, as long as the system's total doesn't surpass it (which it doesn't; section 4.9), since the PSU supplies only as much power as needed.
Drive bays	6 hot-swap (all would have been used)	8 hot-swap (6 used, 2 free; OS drives don't occupy any bays as they use the motherboard's M.2 slots)	For the current system, if reused, all bays would be used, meaning the future expansion requirement wouldn't be covered.
Expansion slots	7 x low-profile	7 x full height	Simplifies searching for components and reduces price as the components can be bought in their regular size, giving a wider and often cheaper choice.
Price	-	About $1,175 (Newegg, n.d.-h),	Total of about $1,337

		rail kit about $63 (Provantage, n.d.), heatsink about $99 (Super Micro Computer, n.d.-h)	

Furthermore, as seen in table 19, the recommended chassis addresses the limitations caused by the previous chassis and ensures the requirements of the recommended system are met, including future expansion requirements imposed by previous sections, which is what the client required (table 1). The cooling ensures the system doesn't overheat while the redundant PSU provides more power than required leaving room for future additions and removing the current system's single point of failure. Although it is not the cheapest option, it is the only one addressing all of these limitations, at a cost that remains within the client's budget (section 4.9).

4.6 Reused Components

Some components were also reused, as per including client's requirement to reuse those that mean fit for the system (table 1). First is the existing 24U rack, which follows the 19-inch EIA-310 standard, which is still the most common for server equipment (Flower, 2025; Yuan, 2026), where the recommended 4U chassis mounts in it with the additional rails. It has 18U free, plus 2U freed when the old server is removed (Table 3), far more than the 4U needed, leaving room for future equipment. Then the UPS, rated at about 1,000 W (table 3), supplies more than the recommended system's estimated 395 W from the wall (section 4.9). It can also run during a power outage, thus giving the server time for a safe shutdown. However, just as with any battery, the condition of the UPS batteries should be checked, as they wear out overtime, and replaced if needed. Finally, the custom PCIe 3.0 x8 DAQ card, delivered with the recent detector upgrade, since it connects the detector to the server (table 3). The recommended system provides a much newer PCIe slot generation (section 4.4), allowing it to run at full speed. Its Linux driver is assumed to work with the recommended OS, as they share the same family of distributions (section 4.4), but this should be confirmed with its manufacturer. The remaining components (processor, memory, storage, RAID card, motherboard, chassis and PSU) are replaced, as explained and justified in sections 4.1 to 4.5.

4.7 System integration

All of the components recommended previously (sections 4.1-4.6) must work together (compatibility has been ensured during recommendation) to receive data from the detector, process it, and forward it out of the system. Data travels over buses, which are the connections between components. A shared bus is when several devices use one bus in turn, requiring an arbiter to decide which device may use it at a time, granting permissions and identifying if the bus is busy, while a point-to-point link joins only two devices, so no sharing or arbitration is needed (Hennessy & Patterson, 2019; Patterson & Hennessy, 2021). Memory is also shared, thus the memory controller (provided by the CPU in the recommended architecture) arbitrates between requests, deciding the order in which the cores and other components access it. Previously for devices to access the memory or data to access devices, the processor had to copy this data and forward it to them, but as technology has evolved, DMA (direct memory access) was invented to take that burden off the CPU, allowing devices to communicate straight with the memory (Tanenbaum & Bos, 2015), where nowadays each component has its own DMA controller (the DAQ card, the SSDs and SATA controller for the HDDs, network controller) sending requests which the memory serves. Since, the DAQ card, network controller and SSDs each have their own device now has its own point-to-point PCIe link to the CPU, devices no longer compete for a shared bus, as the PCI-X cards did on the current board (table 3), and the only shared resource left is memory, which access to is managed by the processor-hosted memory controller, where at the current traffic (4.6 GB/s) uses a fraction (3%) of the 153.6 GB/s available, against 72% on the current system (section 3).

The DAQ card would sit in a PCIe 5.0 x8 slot, which is wired directly to the processor, through which's internal network and memory controller, it can write directly to the RAM by DMA, as standard for high-throughput I/O devices, without involving the cores (Tanenbaum & Bos, 2015). The 28 cores then retrieve the events from RAM, save them in their own cache and filter them, the kept 15% leave through the 10 GbE controller, or go to the drives during an outage through the chipset's SATA ports. The event drives share a SATA controller, containing a DMA, built into the motherboard, where the DMA is dedicated for the storage array, where before any data is written, the processor is used to calculate the parity (section 4.3). This controller is connected to the CPU with its own dedicated link, while the two system SSDs connect directly to the processor's PCIe lanes through the motherboard's M.2 slots. Motherboard's chipset now handles only the slower devices, which consist of the drives, USB port or the BMC, while all faster devices are connected directly (section 4.4). Every link has at least 3.5 times the capacity needed at today's maximum rate, and 1.7 times at the doubled rate (table 20).

Table 20

Link capacity of the recommended system against the requirements and current capacity

Link	Current Load	Future Load	Recommended System			Current System	
			Connection	Capacity	Margin now / future	Capacity (table 3)	Margin now
DAQ card to RAM	2.0 GB/s	4.0 GB/s	PCIe 5.0 x8 slot with limit set by the PCIe 3.0 x8 card	About 8 GB/s	4x / 2x	2.0 GB/s	1x (no margin)
Connection to memory: cores and every component connected to memory through CPU's memory controller (every device by DMA, with event storage having an additional intermediary, the motherboards chipset where every drive shares a SATA controller and a DMA)	4.6 GB/s	9.2 GB/s	4 channels of DDR5-4800	About 153.6 GB/s	33x / 17x	6.4 GB/s	1.4x
Network	300 MB/s	600 MB/s	Onboard 10 GbE (one port)	About 1175 MB/s	3.9x / 2x	118 MB/s	0.4x (doesn't suffice needs)
Event drives (outage only)	300 MB/s	600 MB/s	6 SATA ports on the C741 chipset	About 1040 MB/s	3.5x / 1.7x	315 MB/s	1.05x (no margin)

The memory load (second link) is the total possible bandwidth of memory used simultaneously which consist of: DAQ card's write, core's read, and filtered data's write and read (2.0 + 2.0 + 0.3 + 0.3 = 4.6 GB/s; table 3), doubled at the future rate as done for other rates (like in table 5x). The margin assumes that the facility's switch supports 10 GbE (section 4.4), and margin has been calculated by:

margin = capacity ÷ load. All connection and capacity data has been taken from previous sections, together with the future load introduced for the first time in table 5x (4.1-4.5), whereas the current load was identified at the beginning in table 2.

Furthermore, the processor needs to know when a device has finished a transfer or needs attention without spending its time waiting. Polling is one of the techniques used, where the processor repeatedly sends signals to check each device's status. Despite being simple it wastes processor time checking devices that have nothing to report (Tanenbaum & Bos, 2015). The alternative is interrupts, where the processor performs tasks normally, and when a device needs the processor's attention it sends a signal, allowing the processor to handle the signal and return to its work (Tanenbaum & Bos, 2015). Since the DAQ card delivers data continuously in batches, thus interrupt sent with every batch lets one core acknowledge the new events, while the other cores continue working, followed by assigning the new events to free cores. The current and recommended system differs in their interrupt sending methodologies, as the recommended one uses message-signaled interrupts as mentioned (section 4.4, table 18), handled across the devices, processor and operating system, which need to support this kind of interrupts as in table 21.

Table 21

Support required for message-signaled interrupts by components of the system

Level	Requirement
Devices which send the interrupts, so DAQ card, 10 GbE controller, SSDs, SATA controller	They must be capable of sending message-signaled interrupts, which usually all PCIe devices are designed to do. However, the interrupts are designed in a clever compatible way, where even if a device doesn't have this support, it falls back to the older shared-line method, still working but without its benefits (Nguyen et al., n.d.)
Processor and motherboard	They both must provide the required channels (PCIe), and the interrupt controllers must accept the messages and deliver each to the core it is directed to. The processor must also be able to receive the interrupts, whereas the current board's PCI-X and legacy devices used shared physical lines (Super Micro Computer, Inc., 2007)

Operating system	It handles configuration of the interrupt and handles them through each device's driver. It switches the device to MSI-X if they have it, assigns each interrupt its own signal, decides which core handles it, and runs the code that responds (Nguyen et al., n.d.)

The DAQ card's driver support for the message-signaled interrupts must be confirmed with the manufacturer, however, as mentioned, even without it, the card would still work through the fallback method.

Overall, DMA, point-to-point links and message-signaled interrupts allow data to arrive, be filtered and leave the system simultaneously, while the processor can handle interrupts more efficiently, as they no longer share lines, where the processor's cores can be maximized for filtering rather than moving data and handling interrupts.

4.8 Error Detection, Correction and Maintenance

The recommended system consists of many technologies that allow for detecting, correcting and containing fault, where a single component failure won't cause the server to stop, loose data and beam time. ECC memory corrects memory errors. RAID 6 protects filtered data loss in case data is saved to the storage and two drive failures occur, or one failure with a read error during recovery occurs. RAID 1 ensures a failover for the system drive (SSD) so the system keeps running. Additionally, failures can be found early through automatic health reports and logs, and due to the hot-swap slots, failed drivers or fans can be replaced without stopping the server, which all utilize redundancy as failover. Redundant network port, the outage buffer drives and the UPS ensure resilience through network and power failure. Logs, reports and any live system monitoring can be easily accessed remotely through the BMC, while also providing restart, stop and start capabilities. All listed inside table 22. The current system has no remote monitoring, failover components and spare capacity, thus most of the faults required staff on site and interrupted the server's workflow. However, that system operated on much smaller capacities, which in the recommended system makes faults and error more likely, therefore the maintenance must focus on monitoring rather than repairing failures, as detailed in table 23.

Table 22

Error detection, correction and failover measures

Measure	Component	Description	Section
ECC registered memory	RAM	ECC corrects single-bit and detects double-bit memory errors, and registered memory ensures the control signals are stored in buffers, reducing the electrical load on the memory controller (Kingston Technology, n.d.-b; Mellor, 2022)	4.2
RAID 6	Event drives (HDDs)	Ensures the system is running during two drive failures, or one failure plus a read error during the rebuild	4.3
RAID 1	OS drives (SSDs)	Ensures the system is running if one SSD fails	4.3
Array scrubbing	Event drives	Software RAID periodically reads the whole array of drives to find and repair errors (Layton, 2026), preventing drives from failing	4.3
SMART monitoring	All drives	Each drive reports its own health through their SMART features, allowing for detection of failures early (Thomas-Krenn, n.d.)	4.3
Hot-swap drive bays	Chassis	Allows for replacing failed drives without stopping the server	4.5
Redundant power supplies	Chassis	Allows for a failover and replacement, if one fails, without stopping the server	4.5
Redundant hot-swap fans	Chassis	Allows for replacing without stopping the server and prevents overheating if any fail	4.5
Backup network port	Motherboard	Allows for continuous network connection at the same rate if one fails	4.4
Outage buffer	Event storage	Holds three days of filtered data if it fails to get sent across the network due to failures	4.3
UPS	Rack	Ensures the server can safely shut down during power outages and keep running through short power cuts	4.6
Remote management (BMC)	Motherboard	Monitors metrics and logs hardware event with providing remote access and control over the server	4.4

Table 23

Recommended system maintenance

Task	How	How Often
System health monitoring	BMC alerts for suspicious activities, like recorded statistics from sensors surpass the thresholds (Super Micro Computer, 2020)	Daily (includes automatic alerts)
ECC error logging	Errors are logged by the operating system, and when increased corrected errors are reported, the component should be replaced before it fails	Weekly
Drive health and array checks	System bult into each drive (SMART) produces continuous reports, while RAID has a functionality of array scrubbing, where the drive collections are read and their parity is checked and corrected if corrupted, since it requires reading the whole storage, monthly is recommended	Continuous, with the scrubbing scheduled to monthly
Cooling upkeep	Removing dust from the chassis, fans, heatsink and replacing any worn out fans	8-4 times a year
UPS battery test	Verifying if the batteries aren't worn out and replacing if needed	Yearly (additionally just before the installation of the new system)
Software and firmware updates	Follow up with the OS, BIOS, BMC and other program and firmware updates through monitoring notifications and checking the	Monthly at least, follow up with the releases, and notifications, according to producent's recommendation

	producer's pages periodically	
Monitoring lifecycle and planning upgrades	Warranties of components should be monitored since as they pass, they are more likely to fail, and the budget should account for a possible replacement, for example after 5 years with the current drives. Upgrades should consist of analyzing current system, as capacity can be expanded without replacing components	Yearly, and before upgrades

4.9 Summary

The recommended system costs about $8,809 and draws about 395 W from the wall (table 24), remaining within the budget, power limit and every other requirement (table 25). There is also a set of actions which the facility must complete before installation to ensure successful implementation of the new system, listed in table 26.

In terms of performance and what the recommended system provides, the 28 cores give a predicted speedup of about 11.9x, above the 10-times target, and still a 10.7x if the serial part of the filtering program turns out to be larger than assumed (table 7, section 4.1). Larger private caches make memory access about 3.9 times faster (table 8, section 4.1). As seen in table 20, every link has at least 3.5 times the capacity needed at the current maximum data rate, and at least 1.7 times faster at the predicted future rate, compared to the current system, where some links don't keep up or leave little to no margin. Furthermore, the memory also holds the largest buffer required with a large margin (section 4.2) and the storage suffices a full three-day backlog at the highest requirements, also maintaining plenty headroom (section 4.3), which the network can clear out within a day (Appendix A.1) (where the requirements were already calculated with headroom and accounting for future expansion). Tested in the following sections with a scaled-down version of the filtering program.

Table 24

Cost and estimated power draw of the recommended system

Component	Model	Price	Power at full usage (approx.)
Processor	Intel Xeon Gold 5512U	$1,476 (Intel Corporation, n.d.-c)	185 W (section 4.1)
Memory	4 x 16 GB Samsung DDR5-4800 RDIMM	4 x $183.85 = $735.40 (OEMPCWorld, n.d.-a)	About 20 W (estimated, from a source which states that two larger 128 GB DDR5 server modules were rate at around 19.4 W combined (Nelon, 2026) thus it leaves plenty of room)
Motherboard	Supermicro X13SEI-TF	$891 (Newegg, n.d.-e)	40 W (estimate, including chipset, network and BMC)
Event drives	6 x Seagate Exos X16 16 TB	6 x $625 = $3,750 (Newegg, n.d.-c)	60 W (6 x 10 W) (section 4.3)
System drives	2 x Kingston DC2000B 240 GB	2 x $310 = $620 (eBay, n.d.)	8 W (2 x 4 W) (section 4.3)
Chassis, PSU and fans	Supermicro SC745BAC-R1K23B	$1,175 (Newegg, n.d.-h)	40 W for fans (estimate, including the heatsink fan)
Rail kit	Supermicro CSE-PT26L-B	$63 (Provantage, n.d.)	-
Heatsink	Supermicro SNK-P0090AP4	$99 (Super Micro Computer, Inc., n.d.-h)	Included with the chassis
Operating system	AlmaLinux 9	Free	-
DAQ card (reused)	Custom PCIe 3.0 x8	-	25 W (maximum; Table 3)

Rack and UPS (reused)	-	-	-
Total		**About $8,809**	**About 380 W** (which will need about 395 W from the wall (380 ÷ 0.96, at Titanium efficiency) (section 4.5))

Table 25

Recommended system against client requirements

Constraint	Requirement	Recommended System	Achieved
Budget	About $10,000 maximum for new hardware (given by the facility)	About $8,809	Yes
Power	Under 1,000 W to remain within the UPS rating	About 395 W from the wall needed (table 26)	Yes
Space	Within the rack's free space of 18U with the current server chassis and depth up to 700 mm	Takes 4U of the free 20U while being 647 mm deep (Super Micro Computer, n.d.-g)	Yes
Reuse	Keep the components fit for purpose	Rack, UPS and DAQ card reused (section 4.6), with justification for each component not reused	Yes
Future expansion	Account for possible future detector upgrade's effect on the system	Sized for a doubled data rate reasoned with similar detectors (section 4.1), with spare	Yes

		capacity in every connection (table 26)	
Throughput (requirement established based on the client expecting to use the detector at its maximum intensity in part 4.1)	At minimum 5x the current, with 10x target, accounting for future expansion	About 11.9x predicted (section 4.1), tested in section 7-8	Yes

Table 26

Actions required during migration

Action	Reason	Section
Update the motherboard's BIOS to version 2.1 or later	The 5th-generation processor requires BIOS 2.1 or above (Newegg, n.d.-e; Super Micro Computer, n.d.-c)	4.1
Confirm the switch that the server is connected to, supports 10 GbE, and the analysis facility is able to receive that	The full network speed depends largely on the link to the analysis facility, which is an outside network, outside of the computer system, hence outside the scope of this report	4.4
Check the UPS batteries	The year when the UPS was installed is unknown, and since batteries wear out over time, it should be verified, before the computer system starts relying on it	4.6
Confirm the server room's cooling	The recommended system uses more power, thus generates more heat, if the	4.5

	room doesn't have proper ventilation, the heat may not be able to escape, causing the system to overheat regardless of its cooling	

5. Performance Impact Discussion (200)

The recommended system not only makes the processing faster, but it also makes parallel processing possible for the first time. All of the components add up to make for the improved performance like more cores speeding up the parallel processing or L2 cache speeding up data retrieval for each core. Additionally new connections increasing data transfer rates, and changes which new components have that ensure better performance, that were ignored for keeping outcomes conservative, thus the 28 cores are not left waiting. This design of the system as a whole, is what turns extra cost into extra throughput. All summed up in table 28 compared against other, independent benchmarks, if existent.

Table 28

Performance improvement estimations against independent benchmarks

Component	Estimated Improvement	Benchmark	Comparison	Effect on the client
Processor	11.9x speedup using Amdahl's law with 28 cores and P = 0.95 (table 7)	Recommended processor has a CPU Mark for multithread rating of 59,884 (PassMark Software, n.d.-b), the current has 425 (PassMark Software, n.d.-a) about 141x speedup	The benchmark is calculated without any serial part, thus there is no cap at 20x with 5% serial work, as used for the speedup calculations, thus it shows a far larger gain. It also runs the	Parallelization will be supported for the first time, using the filtering program's multithreading capabilities, removing the bottleneck that limited the facility to a

			tests on all 56 threads which was ignored during the calculation. Nevertheless, a large speedup expected appears as a correct assumption.	lower intensity of the beam setup
Each core	Assumed the same clock speed for all cores in both processors for conservative Amdahl's law estimation (section 4.1), thus not estimated.	Recommended processor has a CPU Mark for single-thread rating of 3,019 (PassMark Software, n.d.-b), the current has 607 (PassMark Software, n.d.-a) about 5x speedup	Even at singular threads, the new CPU is rated higher, suggesting a faster processing speed despite a lower base clock, proving the 11.9x estimate as conservative	The serial part of the program will also run faster; hence, the expected speedup can be expected to be far more than estimated
Cache	Average memory access time was estimated to be about 3.9x lower; thus, faster, from 46.7 to 11.9 ns (table 8)	No benchmark was found, although the estimated access times were estimated using measured latencies of the same core family (Lam, 2023)	The miss rates used to calculate the estimate are worst-case assumption, thus the overall estimate cannot be confirmed directly	Reduction of the average memory access times means each core waits less for data during processing, as the private cache is big enough to store the current working set
Memory bandwidth	Recommended system is 24x	No benchmark was found	The peak is the value	Would provide the system to

	higher (6.4 GB/s against 153.6 GB/s) (section 4.2)		calculated, where real bandwidth is typically lower, but the 9.2 GB/s required with accounting for future expansion is only 6% of the peak (table 20)	send a continuous stream of data, supporting the speed at which data is produced (sent from the detector), thus supporting the amount of data the DAQ sends, and the processor reads and write at the same time, without leaving any component waiting on memory
DAQ card bandwidth	Recommended system is 4x higher, (2 GB/s against 8 GB/s) (table 20)	PCIe is a standard, which provides its estimated bandwidth/speed (Nandra, 2009), and no benchmark was found for this case.	Current system's connection is older than the DAQ card PCIe, so it implies a limit, where now the generation of the connection surpasses that of the card, thus bandwidth is confirmed by the card's own PCIe 3.0 x8 limit	Limit of the bandwidth is set by the DAQ card and not the connection even at a doubled (estimated future) rate

Storage data rate	Write speed is estimated to be about 3.3x higher (315 to about 1,044 MB/s) whereas the capacity about 21x higher (table 16)	No benchmark found	Estimation was made based on the manufacturer's rating for large, sequential writes, which matches how the filtering program writes, however if the actual writes would be smaller than expected, then it would slow it down	Filtered data can be stored safely without slowing down the system or filling up RAM during an outage or a network downtime, even if the writes are smaller than expected, but at the future speed it may be a consideration, without testing it against the real filtering program, it can't be known for sure
Network bandwidth	Increased in the recommended system by about 10x (from 118 to about 1,175 MB/s) (section 4.4)	Ethernet's realistic maximum payload rate, is about 94.8% of its maximum speed (IBM, 2026), which was accounted for during calculations	Estimation was made based on this benchmark, but it also depends on the switch the facility uses and the network that connects PrecisionBeam to the analysis facility, and whether they're able to receive this data	Filtered data can be sent across the network at the same time it is processed, leaving no leftover data waiting to be sent after detector stops sending in data for the day, where, full backlogs,

				if any, are cleared within about a day

Some estimates couldn't have been confirmed against independent benchmarks as they were not found or because they strictly depend on this workload, like the filtering program. Thus, these benchmarks will also be tested in a simulated environment in section 7, directly measuring how throughput scales with the number of threads (imitating cores), the effect of each private cache being too small to hold the output buffer (reasoning behind the L2 cache size and each core having a private output buffer).

6. Alternative Architecture Considered

At each level and component of the recommended system, alternatives were considered, which would require little or no changes to the rest of the system, as the alternatives are usually compatible. However, one bigger alternative that would not change the performance significantly, would be using and AMD processor instead of an intel one. It would be fully compatible as both are x86 architecutre, however they differ in majority of the specifications, and as mentioned in section 4.1, it would slow doesn the parallel portion of the program through forcing part of each core's output buffer to overflow to the lower level memory (the L3 cache). AMD's processors comparable to the Xeon line jump from 24-core to 32-cores, a 24-core would be comparable with the current alternative intel processor identified in section 4.1. At similar specs this option would reduce the performance and speedup but would be a more budget friendly alternative (even up to about half of the current processor's price) (Advance Micro Devices, n.d.), but a different motherboard would be needed.

Some of the smaller alternatives considered throughout, consist of cheaper CPU, which provides the required performance, but contains less cores and is an older generation, but is compatible with the current system, and if the facility decides the latter is more suitable, then no changes will need to be made to the other components. This also applies for configurations of the RAM or storage, where the same model of different capacity can be chosen, ensuring full compatibility. Some alternatives shouldn't be considered due to causing more expenses in any scenario or not being compatible, such as choosing a different processor architecture, because of the restrictions set by

the DAQ card and filtering program which operates on x86. Furthermore, both primary and secondary memory have also been presented a split purchasing approach, where the capacity accounting for rate after future expansion can be incremented with one purchase, and no changes to the system, anytime the facility sees as fit. Overall, the system has presented many alternatives and space for decision, where the recommended system is believed to be the best configuration at the given requirement.

7. Program Overview and Purpose

The benchmark program was designed in a way to imitate the actual program PrecisionBeam uses, as it generates potential event data and passes it through a filtering process which imitates that described by PrecisionBeam. It was written in C++ and with results printed into the terminal. The program initially generates a list of events, where each event contains its own list of readings, and a checksum list, containing the checksum for each event. The value in the checksum list is compared against a newly computed checksum; thus, corrupted events are discarded. Then for each event it compares every reading against a threshold and removes readings which don't suffice it and changes the size of the list to account only for the valid readings. Then the number of elements in each event is compared against another threshold, and if the number of readings doesn't suffice, the event is discarded. The program itself allows for modifying the number of threads used, number of readings, events and thresholds by modifying one variable for each, making it universal and easily adjustable to an experiment's needs.

The purpose of the program is therefore to imitate a single-core and multi-core architecture through threads, to portray the benefits of parallelization. It is also done to demonstrate that increasing number of cores will significantly improve performance, even at a much smaller system. The benchmark calculated will also allow to verify if the estimated serial portion of the program is aligned with reality and to confirm the calculated speedup estimation.

The program initially runs the filtering function on a single thread, one continuous operation which filters event by event, and then a multi-threaded version splits the data across the number of threads decided. Each thread gets assigned a range of events it processes, where it accesses only the positions in the array of its events, thus no lock is needed.

8. Program Performance Analysis

8.1 Methodology

The program couldn't be run on the recommended system, and it was tested on a system with a 12-core ARM processor, running macOS and compiled with Apple clang++ version 21. It was also run on the system while it was plugged into the wall, and no other programs were running. The first experiment runs the filtering function on a single thread, representing the current single core system. The second experiment splits the same generated events array across 12 threads, each performing filtering simultaneously, where each thread represents a core, providing a conservative speedup calculation, like during the calculation of the speedup in section 4.1 (which also neglected the ability of many processors to run two threads per core). Number of events was still 30,000 as established in section 4.1, since using a smaller workload could overstate the serial fraction, where the time cost of creating and joining threads would take a larger share of the shorter run. Since the workload matches the real one, it allows for predicting what the speedup would have been at the actual number of cores.

The size of each reading is assumed to be 12-bit, as it is a common resolution in detector readout (such as in the ATLAS calorimeters; Buchanan et al., 2008), and it is higher than the 7–8-bit readout of the Belle II pixel detector (Knopf et al., 2011), leaving margin for a case where the readings turn out to be larger. The size of each reading was set as 16-bit unsigned integers (0-65,535) because it is the smallest standard type that holds a 12-bit analogue-to-digital value (which the detector is assumed to output). Therefore, corresponding to the estimated 22-67 kB per event (section 4.1), as 20,000 x 2 bytes = 40,000 bytes = 40 kB. It also gives about 1,200 MB in total (30,000 x 40,000 = 1,200,000,000 B = 1,200 MB), which is far larger than the testing system processor's caches, imitating streaming from main memory, just as in the real system.

The program generates events with the same proportions that have been identified to exist in the real data sets (table 2), where 15% of events are designed to pass all tests, thus both tests should keep exactly 4,500 events. This was done through making that 15%, to have a number of valid readings above the second threshold, where each reading, in order to be valid, had to have a value above the first threshold.

Then the program duplicates the two generated lists and runs the experiments one by one with timers present to calculate how long each experiment took, and in the end calculations for speedup and serial portion were done. Since the speedup would only show the improvement of single against 12 cores, the program also calculates the predicted speedup on 28 cores against the calculated serial and parallel proportion, based on the same equation used in section 4.1. The processing speed is also calculated, considering each event has a size within the range identified, resembling the realistic size of the data. Additionally, the program was run 5 times with recompiling it between every run, documented in section 4.2, and the detailed source code can be found in Appendix A.2

8.2 Testing Results

The outputs of each of the 5 trials can be found in Appendix A.1, and the results are summarized in table 29. In each of the trials the program kept 4,500 events from the 30,000, ensuring the outcome stayed the same, on both single and multiple threads.

Table 29

Program output results table with averages

Trial	Single-threaded execution time (s)	Multi-threaded execution time (12 threads) (s)	Processing speed during multi-threaded execution (MB/s)	Speedup	Serial portion calculated	Parallel portion calculated	Predicted speedup at 28 cores using the calculated values
1	0.521392	0.061377	19,551.3	8.49491x	3.75101%	96.249%	13.9112x
2	0.516833	0.058671	20,453	8.80899x	3.29313%	96.7069%	14.8215x
3	0.510929	0.0539095	22,259.5	9.47771x	2.41935%	97.5806%	16.9366x
4	0.510926	0.0528248	22,716.6	9.67209x	2.18803%	97.812%	17.6016x
5	0.516588	0.0575182	20,862.9	8.98129x	3.05555%	96.9444%	15.3425x
Average	0.5153	0.05686	21,168.7	9.09x	2.94%	97.06%	15.72x

The processing speed was calculated by dividing the amount of total data processed over the time it was processed at (1,200 MB÷ execution time). Speedup (S) was by dividing the single-threaded execution time by the multi-threaded execution time. Serial portion (f) was calculated by solving Amdahl's law, $S = 1 \div (f + (1 - f) \div N)$, for f, giving $f = (N \div S - 1) \div (N - 1)$ with N = 12 threads (Amdahl, 1967). The parallel portion is 1 − f. The predicted speedup at 28 cores was calculated with Amdahl's law with the given $1 \div (f + (1 - f) \div 28)$, using the serial portion from the same trial. Averages are the means of the five trials.

The single-threaded execution time differed slightly across trials with a range between 0.511s and 0.521s. The multi-threaded time execution time varied slightly more, between 0.053s and 0.061s. This is expected, because at runs this short, the background processes affect the outcomes more. On average, the execution time decreased from 0.515s to 0.057s, at about 9 times faster, with an average processing rate of 21,168.7 during the multi-threaded runs. The speedup ranged from 8.49x to 9.67x, with an average of 9.09x. The serial portion, calculated for each run based on that particular run's execution time and the 12 threads used, imitating 12 cores, averaged 2.94%, ranging from 2.19% to 3.75%, which gave an average parallel portion of 97.06% and an average predicted speedup for 28 cores of 15.72x. These results are compared with the earlier predictions in section 8.3.

8.3 Benchmarking, Analysis and Evaluation

In section 4.1, the number of cores was chosen using Amdahl's law with an assumed parallel proportion of 0.95. The benchmark calculated in section 8.2 makes it possible to compare this assumption against measured results. Benchmark calculates both the serial and parallel portion of the processing for each trial, based on the speedup, later used to recalculate the speedup if this same set-up ran on a 28-core machine, and with 28 threads, which are all compared against the assumption made during choosing the recommended system in table 30.

Table 30

Estimates from section 4.1 compared against the benchmark results

Value	Estimated in section 4.1	Benchmark
Serial portion	5% (1% and 6% were also tested in table 7)	2.94%
Parallel portion	95%	97.06%
Maximum possible speedup (1÷ (1-P) where P is the parallel portion thus equivalent to 1 ÷ serial portion)	20x	34x
Predicted speedup on 28 cores	11.9x	15.7x

The measured serial portion of 2.94% lies between the 1% approximated by PrecisionBeam and the 5% assumed in section 4.1 and about half of the 6% worst-case scenario in table 7. Real overheads, therefore, made the serial part larger than what PrecisionBeam approximated, as expected, but stayed well within what the margin allowed. However, there are certain limitations which reduce the accuracy of the results, as explored in table 31.

Using the value from the benchmark, speed-up for a 28-core processor for the exact same conditions could be predicted using Amdahl's law (Amdahl, 1967), which gives a speedup of about 15.7x. This is much more than the predicted 11.9x estimated in table 7, but since the testing system largely differs from the recommended one and many limitations have been identified (see table 31), an unequivocal conclusion cannot be made. However, it still showcases how much a parallel processing scenario can increase the processing rate, thus the change proposed at first, to increase the number of cores, for PrecisionBeam is justified. Additionally, the result, although not perfect, indicates the improvement was very successful, as a speedup of 9.09x on 12 threads more than 3/4 th's of the ideal 11.9x.

Table 31

Limitations of the benchmark

Limitation	Description	Effect
Different hardware	The benchmark program ran on a 12-core ARM processor laptop, not on the recommended 28-core server processor.	The speedup, both estimated in section 4.1 and the predicted one from the benchmark, cannot be confirmed.
Simplified data filtering	Despite the event size and the proportions of kept and corrupted events matching the real data, they are missing the patterns found in real detector data. The filter is thus much simpler, as it is based only on thresholds instead of more advanced pattern identification, calculations or mapping.	Simple filtering process causes the filtering by itself to take less time; thus, processing takes much less time. Since filtering is the parallel portion of the program, it makes it be relatively shorter, compared to the real program, as fixed time costs such as creating and joining threads, make up a larger share of the run time.

Uniform events and distribution	All events were set to have a size of 40 kB and were divided between threads once. In reality there isn't only one set of data, and events come in continuously, where events differ in sizes, causing some to be processed longer, and some shorter.	This could be a potential source of lengthening the serial portion of the program, as some cores wait for others, or are allocated more events, depending how the actual program allocates events, clears buffers, and stores events.
Threads imitating cores	12 threads run were selected to imitate the 12 cores available on the testing system. The macOS running on the testing system decides where each thread is run, thus there's no certainty that each thread was run on a separate core. Additionally, the cores may have been shared with background processes or even two threads could share a core at some point.	Some threads may have run more slowly than others, so the faster ones finished early and waited. This would lower the measured speedup and make the serial portion appear higher than it is.
Program differences	The program is meant to measure performance from parallel gain, thus it doesn't recreate the output stage, with output buffers per core, handing over full buffers, or where the buffers sit in the memory hierarchy. Considering the architecture of the testing system doesn't resemble that of the recommendation, the missing parts, would not produce outputs like on the recommended system. Additionally, the source code, nor the specifications of the architecture of the actual filtering program are not provided, thus creating a program which would resemble that one, one-to-one is impossible.	Both the parallel and serial portion could differ significantly, as an output buffer could reduce the serial portion by pre-allocating each core its own space to save events, instead of taking turns writing to a shared structure. but it could also increase it, due to buffer handover costs (moving data to slower, lower-level memory). The benefit of the L2 keeping all buffers couldn't be tested.

Overall, most of these limitations could only be removed by running the benchmark on the actual system, with the actual filtering program doing the filtering. However, the results support that increasing cores, and allowing PrecisionBeam's filtering program to

run its multi thread capability would be successful at significantly increasing the speed at which the data is filtered.

9. Conclusion

In conclusion, the analysis showed that PrecisionBeam's system can't support the detector at full intensity, because of the single core processor, not because of a lack of memory, as previously recommended. The data rate sent to the system exceeded the processing rate of the processor, which is why data started bottling up in the RAM, and the whole system became overwhelmed. The issue was rooted in events being processed one at a time, since there exists no processor with clock rate high enough to provide the desired rate. The proposed system replaces the single core CPU to a 28-core one with the same x86 architecture, together with a faster memory, storage and network connections. RAM size wasn't identified as the root cause of the issue, however, its capacity also wasn't enough to support the amount of data sent. All of the new components would cost about $8,809 and draw about 395 W, which is within the clients $10,000 and 1,000 W budget. Amdahl's law predicted a speedup of above 10x for the proposed processor, which, together with other system specifications, like the storage size, would account for future upgrades, bursts, fluctuations and downtimes. The benchmark program supported the estimate with a 9.09x speedup on 12 threads with a serial portion of about 2.94%, well within the 5% and 6% accounted for during processor selection. Despite the program's limitations, the results suggests that the recommended system would support the detector to run at full intensity again and fulfill PrecisionBeam's requirements.

10. References

1. Aaij, R., Albrecht, J., Belous, M., Billoir, P., Boettcher, T., Brea Rodríguez, A., vom Bruch, D., Cámpora Pérez, D. H., Casais Vidal, A., Craik, D. C., Fernandez Declara, P., Funke, L., Gligorov, V. V., Jashal, B., Kazeev, N., Martínez Santos, D., Pisani, F., Pliushchenko, D., Popov, S., . . . Zhuo, Y. (2020). Allen: A high-level trigger on GPUs for LHCb. Computing and Software for Big Science, 4, Article 7. https://doi.org/10.1007/s41781-020-00039-7
2. Adamczyk, K., Aggarwal, L., Aihara, H., Aziz, T., Babu, V., Bacher, S., Bahinipati, S., Bari, M., Baroncelli, Ti., Baroncelli, To., Bassi, G., Batignani, G., Baudot, J., Bauer, A., Behera, P. K., Bergauer, T., Bertacchi, V., Bettarini, S., Bilka, T., . . .

Zhang, T. (2022). *The design, construction, operation and performance of the Belle II silicon vertex detector*. Journal of Instrumentation, 17(11), Article P11042. https://doi.org/10.1088/1748-0221/17/11/P11042

3. Advanced Micro Devices. (n.d.). *AMD EPYC 8224P*. Retrieved October 13, 2026, from https://www.amd.com/en/products/processors/server/epyc/4th-generation-9004-and-8004-series/amd-epyc-8224p.html
4. Amazon Web Services. (n.d.). *What's the difference between an SSD and a hard drive?* https://aws.amazon.com/compare/the-difference-between-ssd-hard-drive/
5. AMCC. (2005). *3ware 9550SXU SATA II RAID controller*. Retrieved October 13, 2026, from https://www.oicp.fr/images/Image/File/Cartes%20dextension/Carte%20controlleur%20RAID/OICP-3WARE_%209550SXU.pdf
6. Amdahl, G. M. (1967). *Validity of the single processor approach to achieving large scale computing capabilities*. In Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (pp. 483–485). ACM. https://doi.org/10.1145/1465482.1465560
7. Mellor, C. (2022, May 5). *RDIMM*. Blocks & Files. https://blocksandfiles.com/2022/05/05/rdimm/
8. Buchanan, N. J., et al. (2008). *Design and implementation of the Front-End Board for the readout of the ATLAS liquid argon calorimeters*. Journal of Instrumentation, 3(03), P03004. https://doi.org/10.1088/1748-0221/3/03/P03004
9. Dell Technologies. (2026). *Cut costs, not performance with single-socket PowerEdge*. Dell Technologies Blog. Retrieved October 1, 2026, from https://www.dell.com/en-us/blog/cut-costs-not-performance-with-single-socket-poweredge/
10. Nelon, J. (2026, May 24). *Micron starts sampling 256GB DDR5 RDIMM modules for AI and server workloads*. Digital Citizen. https://www.digitalcitizen.life/micron-starts-sampling-256gb-ddr5-rdimm-modules-for-ai-and-server-workloads/
11. Drepper, U. (2007, November 21). *What every programmer should know about memory*. Red Hat, Inc. https://people.freebsd.org/~lstewart/articles/cpumemory.pdf
12. eBay. (n.d.-a). *Kingston DC2000B SEDC2000BM8/240G 240 GB Kingston SSD (item 398297803170)*. Retrieved October 6, 2026, from https://www.ebay.com/itm/398297803170
13. eBay. (n.d.-b). *SuperMicro AOC-STG-I2T 2-port 10GbE standard adapter (Item 322131998881)*. Retrieved October 7, 2026, from https://www.ebay.com/itm/322131998881

14. GPUGuru. (2026, July 5). *RAID 5 vs RAID 6: How many drives can fail before you lose data?* EveZone. https://evezone.evetech.co.za/quick-bytes/raid-5-vs-raid-6-how-many-drives-can-fail-before-you-lose-data
15. Exxact. (n.d.). *Samsung M321R2GA3BB6-CQK 16 GB DDR5-4800 memory*. Retrieved October 4, 2026, from https://www.exxactcorp.com/Samsung-M321R2GA3BB6-CQK-E5415707
16. Fermilab & CERN. (2022, December 7). *Fermilab/CERN recommendation for Linux distribution*. Scientific Linux. https://scientificlinux.org/category/uncategorized/fermilab-cern-recommendation-for-linux-distribution/
17. Flower, A. (2025, August 2). *The history of the 19-inch rack. MACARAC 19" Inch Sheetmetal*. https://www.macarac.com.au/blogs/news/the-history-of-the-19-inch-rack
18. Fusion Worldwide. (2026). *GPU shortage and price increases in 2026*. Retrieved October 7, 2026, from https://info.fusionww.com/blog/gpu-shortage-and-price-increases-in-2026
19. GeeksforGeeks. (2025, July 23). *Types of cache*. https://www.geeksforgeeks.org/system-design/types-of-cache/
20. Geßler, T., Kühn, W., Lange, J. S., Liu, Z., Münchow, D., Spruck, B., & Zhao, J. (2015). *The ONSEN data reduction system for the Belle II pixel detector. IEEE Transactions on Nuclear Science, 62*(3), 1149–1154. https://doi.org/10.1109/TNS.2015.2414713
21. Getprice.pl. (n.d.). *Karta sieciowa Intel X520-DA2 2x SFP+ 10GbE (AMZX520DA2) wysoki profil* [Intel X520-DA2 network card, 2 x SFP+ 10GbE, full height]. Retrieved October 7, 2026, from https://getprice.pl/pl/karta-sieciowa-intel-x520-da2-2x-sfp-10gbe-amzx520da2-wysoki-profil-37356.html
22. Štefan. (2025, January 3). *Low-profile gaming GPUs: Performance in a small package*. GrafickeKarty.sk. Retrieved October 8, 2026, from https://grafickekarty.sk/en/gpu-tips/low-profile-gaming-gpus/
23. Gregg, C., & Hazelwood, K. (2011). *Where is the data? Why you cannot debate CPU vs. GPU performance without the answer*. In 2011 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS) (pp. 134–144). IEEE. https://doi.org/10.1109/ISPASS.2011.5762730
24. Hennessy, J. L., & Patterson, D. A. (2019). *Computer architecture: A quantitative approach (*6th ed.). Morgan Kaufmann.
25. Herlihy, M., Shavit, N., Luchangco, V., & Spear, M. (2021). *The art of multiprocessor programming* (Second edition). Elsevier Science & Technology.
26. Huber, S., Konorov, I., Levit, D., Paul, S., & Steffen, D. (2021). *Performance of the data-handling hub readout system for the Belle II pixel detector*. IEEE Transactions on Nuclear Science, 68(8), 1961–1967. https://doi.org/10.1109/TNS.2021.3083720

27. Hyperscalers. (n.d.). *DDR5 Samsung 4800 16GB RDIMM M321R2GA3BB6-CQK*. Retrieved October 4, 2026, from https://www.hyperscalers.com/RAM-Memory-DDR5-16GB-RDIMM-4800-Samsung-M321R2GA3BB6-CQK
28. IBM. (2026, April 8). *Adapter performance guidelines*. IBM AIX 7.2 documentation. Retrieved October 1, 2026, from https://www.ibm.com/docs/ssw_aix_72/performance/adapter_perf_guides.html
29. Intel Corporation. (n.d.-a). *Intel Xeon Silver 4516Y+ processor (45M cache, 2.20 GHz): Product specifications*. Retrieved September 29, 2026, from https://www.intel.com/content/www/us/en/products/sku/237556/intel-xeon-silver-4516y-processor-45m-cache-2-20-ghz/specifications.html
30. Intel Corporation. (n.d.-b). *Intel Xeon Gold 5412U processor (45M cache, 2.10 GHz): Product specifications*. Retrieved October 1, 2026, from https://www.intel.com/content/www/us/en/products/sku/232374/intel-xeon-gold-5412u-processor-45m-cache-2-10-ghz/specifications.html
31. Intel Corporation. (n.d.-c). *Intel Xeon Gold 5512U processor (52.5M cache, 2.10 GHz): Product specifications*. Retrieved October 1, 2026, from https://www.intel.com/content/www/us/en/products/sku/237565/intel-xeon-gold-5512u-processor-52-5m-cache-2-10-ghz/specifications.html
32. Intel Corporation. (2004a). *Intel E7520/E7320/E7525 memory controller hub (MCH) thermal/mechanical design guide (Document No. 302403-003*). https://www.intel.com/content/dam/doc/design-guide/e7520-e7320-e7525-memory-controller-hub-guide.pdf
33. Intel Corporation. (2004b). *Intel Xeon processor with 800 MHz system bus datasheet (Document No. 302355-001*). Retrieved October 13, 2026, from https://www.manualslib.com/manual/697401/Intel-Xeon.html
34. Intel Corporation. (2021). *Product Brief Intel Optane SSD P1600X: Meet the new, data center-optimized SSDs for boot, logging, and caching*. https://www.rutronik.com/fileadmin/rutronik/Micropages/Intel/Update_2021/data-center-optimized-ssds-for-boot-logging-caching-product-brief.pdf
35. Intel Corporation. (2023). *Intel® 64 and IA-32 architectures software developer's manual: Vol. 3A*. System programming guide, part 1. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
36. Itoh, R., Braun, N., Li, C., Nakao, M., Yamada, S., Suzuki, S. Y., Zhou, Q., Konno, T., Liu, Z.-A., Zhao, J., Hartbrich, O., Park, S.-H., Guan, Y., Lautenbach, K., Reiter, S., & Spruck, B. (2020). *The performance of Belle II high level trigger in the first physics run*. EPJ Web of Conferences, 245, Article 01040. https://doi.org/10.1051/epjconf/202024501040
37. Keysight Technologies. (2020, March 30). *PCIe standards: What you need to know.* Retrieved October 1, 2026, from

https://www.keysight.com/blogs/en/inds/2020/03/30/pcie-standards-what-you-need-to-know

38. Kingston Technology. (n.d.-a). *DDR4 and DDR5 memory population rules based on chipset and processor pairs*. Retrieved October 4, 2026, from https://www.kingston.com/en/memory/memory-population-rules
39. Kingston Technology. (n.d.-b). *DDR5 memory standard: An introduction to the next generation of DRAM module technology*. Retrieved October 4, 2026, from https://www.kingston.com/en/blog/pc-performance/ddr5-overview
40. Kingston Technology. (n.d.-c). *DC2000B PCIe 4.0 NVMe M.2 SSD: Enterprise class for server applications*. Retrieved October 4, 2026, from https://www.kingston.com/datasheets/SEDC2000B_us.pdf
41. Knopf, J., Fischer, P., Kreidl, C., & Perić, I. (2011). *A 256 channel 8-bit current digitizer ASIC for the Belle-II PXD*. Journal of Instrumentation, 6(01), C01085. https://doi.org/10.1088/1748-0221/6/01/C01085
42. Lam, C. (2022, November 8). *AMD's Zen 4, part 2: Memory subsystem and conclusion*. Chips and Cheese. Retrieved October 8, 2026, from https://chipsandcheese.com/p/amds-zen-4-part-2-memory-subsystem-and-conclusion
43. Lam, C. (2023, March 12). *Sapphire Rapids: Golden Cove hits servers. Chips and Cheese*. Retrieved October 1, 2026, from https://chipsandcheese.com/p/a-peek-at-sapphire-rapids
44. Layton, J. (2026, May 11). *Linux software RAID. ADMIN Magazine*. Retrieved October 4, 2026, from https://www.admin-magazine.com/HPC/Articles/Linux-Software-RAID
45. Lee, S., Itoh, R., Higuchi, T., Nakao, M., Suzuki, S. Y., & Won, E. (2012). *Belle-II high level trigger at SuperKEKB. Journal of Physics: Conference Series*, 396(1), Article 012029. https://doi.org/10.1088/1742-6596/396/1/012029
46. LHCb Collaboration. (2015). *LHCb detector performance. International Journal of Modern Physics* A, 30(7), Article 1530022. https://doi.org/10.1142/S0217751X15300227
47. Marr, D. T., Binns, F., Hill, D. L., Hinton, G., Koufaty, D. A., Miller, J. A., & Upton, M. (2002). *Hyper-Threading Technology architecture and microarchitecture. Intel Technology Journal*, 6(1), 4–15. https://www.intel.com/content/dam/www/public/us/en/documents/research/2002-vol06-iss-1-intel-technology-journal.pdf
48. McKenney, P. E. (2025, December 18). *Is parallel programming hard, and, if so, what can you do about it?* kernel.org. https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook-1c.2025.12.18a.pdf

49. Moore, G. E. (1965). Cramming more components onto integrated circuits. *Proceedings of the IEEE, 86*(1), 82–85. https://ieeexplore.ieee.org/document/658762
50. Nandra, N. (2009). *PCI Express: The evolution to 8.0 GT/s [Conference presentation]*. Design Automation Conference 2009. https://www.chipestimate.com/dac2009/presentations/synopsys2.pdf
51. Need to Know IT. (2026, September 6). *Is RAID 5 still safe with large drives? (2026 honest assessment)*. Retrieved October 4, 2026, from https://needtoknowit.com.au/blog/is-raid-5-still-safe-with-large-drives/
52. Newegg. (n.d.-a). *Seagate Exos 7E10 ST8000NM024B 8 TB hard drive - internal*. Retrieved October 6, 2026, from https://www.newegg.com/p/1Z4-002P-024E7
53. Newegg. (n.d.-b). *Seagate Exos X16 ST12000NM003G 12TB hard drive - internal*. Retrieved October 6, 2026, from https://www.newegg.com/p/N82E16822185067
54. Newegg. (n.d.-c). *Seagate Exos X16 ST16000NM001G 16TB hard drive - internal*. Retrieved October 6, 2026, from https://www.newegg.com/p/1Z4-002P-025V3
55. Newegg. (n.d.-d). *Seagate Exos X20 ST20000NM007D 20TB hard drive for enterprise storage* (7200 RPM). Retrieved October 6, 2026, from https://www.newegg.com/p/N82E16822185011
56. Newegg. (n.d.-e). *Supermicro X13SEI-TF server motherboard (5th Gen/4th Gen Intel Xeon Scalable processors, single socket LGA-4677)*. Retrieved October 7, 2026, from https://www.newegg.com/p/2U3-000V-00025
57. Newegg. (n.d.-f). *Supermicro X13SEI-F Intel Xeon server motherboard*. Retrieved October 7, 2026, from https://www.newegg.com/p/N82E16813183824
58. Newegg. (n.d.-g). *Supermicro X13SEW-TF server motherboard*. Retrieved October 7, 2026, from https://www.newegg.com/p/N82E16813183851
59. Newegg. (n.d.-h). *SUPERMICRO SuperChassis CSE-745BAC-R1K23B 4U tower server chassis*. Retrieved October 7, 2026, from https://www.newegg.com/supermicro-cse-745bac-r1k23b/p/N82E16816139348
60. Newegg. (2008, February 25). *SUPERMICRO CSE-822T-400LPB black 2U rackmount server case 400W power supply w/ PFC 1 external 5.25" drive bays*. https://www.newegg.com/black-supermicro-cse-822t-400lpb/p/N82E16811152109?srsltid=AU7gw4VG3Ki1SULHPwIgZNiQ15y6g8tYv0mIiOTebAd_Ak1TSMz4gGB-
61. Nguyen, T. L., Silbermann, M., & Wilcox, M. (n.d.). *The MSI driver guide HOWTO (Linux Kernel Documentation)*. The Linux Kernel Organization. https://docs.kernel.org/PCI/msi-howto.html
62. OEMPCWorld. (n.d.-a). *Samsung 16GB M321R2GA3BB6-CQK DDR5-4800 RDIMM PC5-38400 ECC*. Retrieved October 4, 2026, from https://oempcworld.com/products/samsung-16gb-m321r2ga3bb6-cqk-ddr5-4800-rdimm-pc5-38400-ecc

63. OEMPCWorld. (n.d.-b). *Samsung 32GB M321R4GA3BB6-CQK DDR5-4800 RDIMM PC5-38400 ECC*. Retrieved October 4, 2026, from https://oempcworld.com/products/samsung-32gb-m321r4ga3bb6-cqk-ddr5-4800-rdimm-pc5-38400-ecc
64. ORM Systems. (2026, July 1). *HDD vs. SSD: Speed, price, and lifespan compared: Which should you buy?* https://ormsystems.com/blogs/hdd-vs-ssd-key-differences-performance-and-which-one-to-choose
65. PassMark Software. (n.d.-a). *Intel Xeon 3.00GHz*. CPU Benchmarks. Retrieved October 1, 2026, from https://www.cpubenchmark.net/cpu.php?cpu=Intel+Xeon+3.00GHz&id=1318
66. PassMark Software. (n.d.-b*). Intel Xeon Gold 5512U*. CPU Benchmarks. Retrieved October 8, 2026, from https://www.cpubenchmark.net/cpu.php?cpu=Intel+Xeon+Gold+5512U&id=5955
67. Patterson, D. A., Gibson, G., & Katz, R. H. (1988, June 1). A case for redundant arrays of inexpensive disks (RAID). *In Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data* (pp. 109–116). ACM. https://doi.org/10.1145/50202.50214
68. Patterson, D. A., & Hennessy, J. L. (2021). *Computer organization and design: the hardware software interface (*Second edition.). Elsevier.
69. Paxson, V., Allman, M., Chu, J., & Sargent, M. (2011, June). *Computing TCP's retransmission timer (RFC 6298)*. Internet Engineering Task Force. https://doi.org/10.17487/RFC6298
70. PCI-SIG. (2013). *PCI Express card electromechanical specification (*Rev. 3.0). PCI Special Interest Group. https://pcisig.com/specifications
71. Provantage. (n.d.). *Supermicro CSE-PT26L-B 4U chassis rail kit*. Retrieved October 8, 2026, from https://www.provantage.com/supermicro-cse-pt26l-b~7SUPA00J.htm
72. Red Hat. (n.d.-a). *Interactively installing RHEL from installation media: Chapter 1. System requirements and supported architectures*. Red Hat Enterprise Linux 9 documentation. Retrieved October 1, 2026, from https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/interactively_installing_rhel_from_installation_media/system-requirements-and-supported-architectures_rhel-installer
73. Red Hat. (n.d.-b). *Automatically installing RHEL. Red Hat Enterprise Linux 9 documentation*. Retrieved October 4, 2026, from https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html-single/automatically_installing_rhel/index
74. Schnepf, M. J., Fischer, M., & Petzold, A. (2024). *CPU performance study for HEP workloads with respect to the number of single-core slots*. EPJ Web of Conferences, 295, Article 07010. https://doi.org/10.1051/epjconf/202429507010

75. Schuchart, J., Hackenberg, D., Schöne, R., Ilsche, T., Nagappan, R., & Patterson, M. K. (2016). *The shift from processor power consumption to performance variations: Fundamental implications at scale*. Computer Science – Research and Development, 31(4), 197–205. https://arxiv.org/abs/1808.08106
76. Seagate Technology LLC. (2007a). *Barracuda 7200.11 Serial ATA product manual* (Rev. A, Publication No. 100452348). https://www.seagate.com/staticfiles/support/disc/manuals/desktop/Barracuda%207200.11/100452348a.pdf
77. Seagate Technology LLC. (2007b). *Barracuda 7200.10 Serial ATA product manual* (Rev. K, Publication No. 100402371). https://www.seagate.com/staticfiles/support/disc/manuals/desktop/Barracuda%207200.10/100402371k.pdf
78. Seagate Technology LLC. (2019). *Exos X16 data sheet*. https://www.seagate.com/www-content/datasheets/pdfs/exos-x16-DS2011-1-1904US-en_US.pdf
79. Serverblink. (n.d.). *KVR400D2S4R3/1G: Kingston 1GB DDR2-400MHz PC2-3200 ECC registered CL3 240-pin DIMM single rank memory module*. Retrieved October 1, 2026, from https://www.serverblink.com/kvr400d2s4r3-1g-kingston-technology-kingston-1gb-ddr2-400mhz-pc2-3200-ecc-registered-cl3-240-pin-dimm-single-rank-memory-module/
80. ServeTheHome. (2026). *Buying servers in 2026 is tough but here are our buyer's tips* (p. 2). Retrieved October 1, 2026, from https://www.servethehome.com/buying-servers-in-2026-buyers-guide-tips-amd-dell-hpe-asrock-rack-asus-supermicro/2/
81. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.
82. Super Micro Computer. (n.d.-a). *SC822 chassis series user's manual* (Rev. 2.0). https://www.supermicro.com/manuals/chassis/2U/SC822.pdf
83. Super Micro Computer. (n.d.-b). *CPU information: 5th Gen Intel Xeon processors*. https://www.supermicro.com/en/support/resources/cpu-5th-gen-intel-xeon-scalable
84. Super Micro Computer. (n.d.-c). *X13SEI-TF.* Retrieved October 4, 2026, from https://www.supermicro.com/en/products/motherboard/x13sei-tf
85. Super Micro Computer. (n.d.-d). *X13SEI-F.* Retrieved October 4, 2026, from https://www.supermicro.com/en/products/motherboard/x13sei-f
86. Super Micro Computer. (n.d.-e). *X13SEW-TF.* Retrieved October 4, 2026, from https://www.supermicro.com/en/products/motherboard/x13sew-tf
87. Super Micro Computer. (n.d.-f). *Supermicro data center server, blade, data storage, AI system*. Retrieved October 6, 2026, from https://www.supermicro.com/en/

88. Super Micro Computer. (n.d.-g). *SuperChassis 745BAC-R1K23B*. Retrieved October 7, 2026, from https://www.supermicro.com/en/products/chassis/4u/745/sc745bac-r1k23b
89. Super Micro Computer. (n.d.-h). *Supermicro 4U active CPU heat sink socket LGA4677 (SNK-P0090AP4)*. Supermicro eStore. Retrieved October 7, 2026, from https://store.supermicro.com/us_en/4u-active-cpu-cooler-x13-snk-p0090ap4.html
90. Super Micro Computer. (n.d.-i). *Supermicro 2U passive CPU heat sink socket LGA4677 (SNK-P0088P)*. Supermicro eStore. Retrieved October 12, 2026, from https://store.supermicro.com/2u-passive-cpu-cooler-x13-snk-p0088p.html
91. Super Micro Computer. (2007). *X6DH8-XG2/X6DHE-XG2 user's manual* (Rev. 1.1a). https://www.supermicro.com/manuals/motherboard/E7520/MNL-0737.pdf
92. Super Micro Computer. (2020). *BMC IPMI: Intelligent Platform Management Interface user's guide*. https://www.supermicro.com/manuals/other/IPMI_Users_Guide.pdf
93. Super Micro Computer. (2024). *SUPER X13SEI-TF/X13SEI-F user's manual* (Manual No. MNL-2450). https://www.supermicro.com/manuals/motherboard/X13/MNL-2450.pdf
94. SUSE. (2015). *System analysis and tuning guide: Chapter 14. Tuning the memory management subsystem (openSUSE Leap 42.1)*. Retrieved October 1, 2026, from https://doc.opensuse.org/documentation/leap/archive/42.1/tuning/html/book.sle.tuning/cha.tuning.memory.html
95. Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.
96. Thomas-Krenn. (2020, May 5). *SMART Attributes documentation*. Retrieved October 8, 2026, from https://www.thomas-krenn.com/en/wiki/SMART
97. Mpitziopoulos, A. (2018, April 12). *Tom's Explains: What do 80 PLUS Bronze, Silver, Gold & Titanium signify?*. Tom's Hardware https://www.tomshardware.com/news/what-80-plus-levels-mean,36721.html
98. USB Implementers Forum. (2000). *Universal serial bus specification* (Rev. 2.0). https://www.usb.org/document-library/usb-20-specification
99. Vishnu, M. (2025, June 3). *Understanding queueing theory*. BigBinary. https://bigbinary.com/blog/understanding-queueing-theory
100. Wielers, M. (2014). *Introduction to data acquisition* [Lecture slides]. Rutherford Appleton Laboratory. https://indico.cern.ch/event/354290/contributions/833024/attachments/702126/963973/daq_ral_lecture_041214.pdf
101. Wikipedia contributors. (2026, August 13). *Amdahl's law*. In Wikipedia. https://en.wikipedia.org/wiki/Amdahl%27s_law

102. Yuan, K. (2026, April 30). *From factory floor to data center: The cross-industry journey of a 19-inch industrial cabinet*. TPS. https://tps-elektronik.com/en/from-factory-floor-to-data-center-the-cross-industry-journey-of-a-19-inch-industrial-cabinet/
103. Zijlstra, P. (2023). *High memory handling (Linux Kernel Documentation v6.3.0)*. The Linux Kernel Organization. https://www.kernel.org/doc/html/v6.3/mm/highmem.html

AI Acknowledgment Table

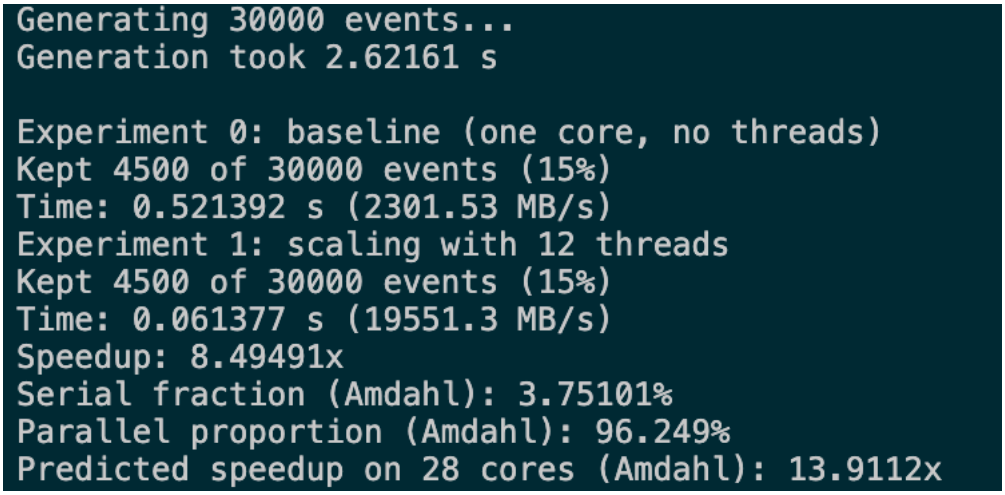
AI tool	Use Case
Claude Sonnet 5.5	Feedback regarding language and improvements.

11. Appendix A

11.1 Appendix A.1 – Program Execution Output

Figure 5

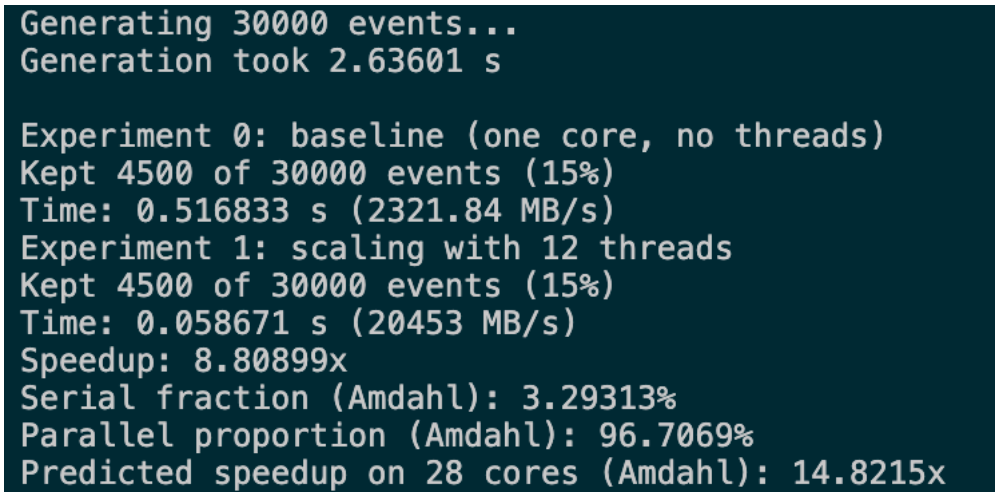
Trail 1 output of the benchmark program



Taken screenshot of the output in the console

Figure 6

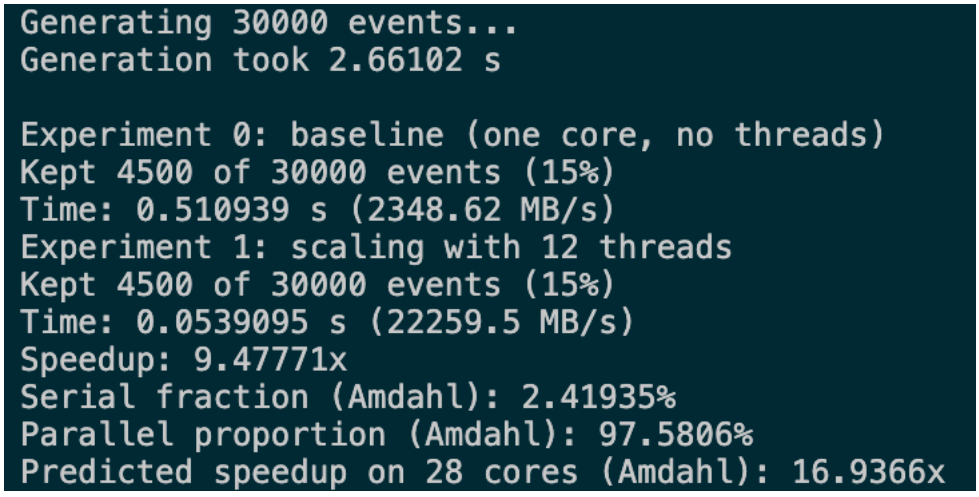
Trail 2 output of the benchmark program



Taken screenshot of the output in the console

Figure 7

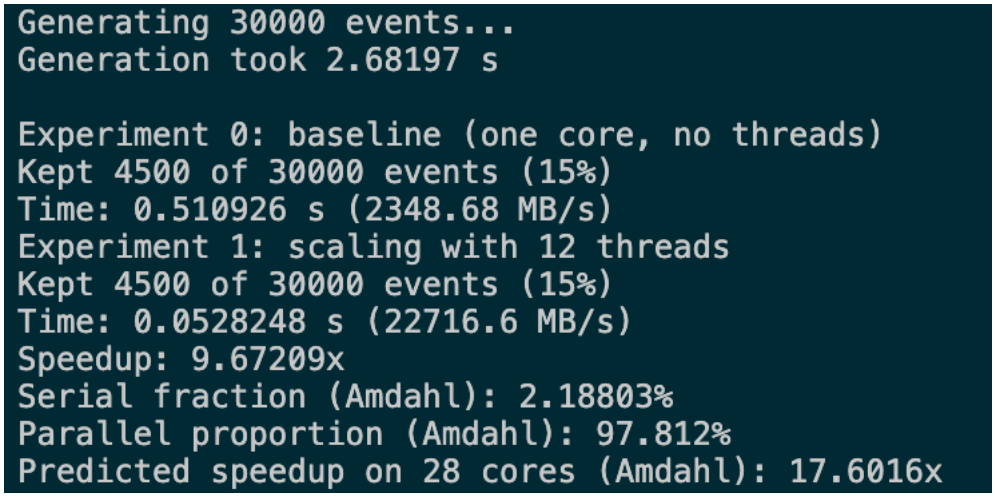
Trail 3 output of the benchmark program



Taken screenshot of the output in the console

Figure 8

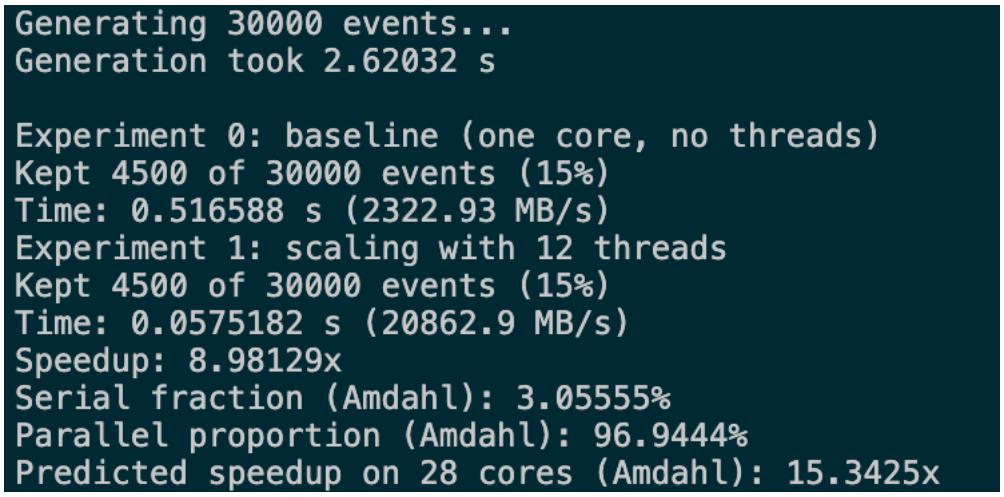
Trail 2 output of the benchmark program



Taken screenshot of the output in the console

Figure 9

Trail 2 output of the benchmark program

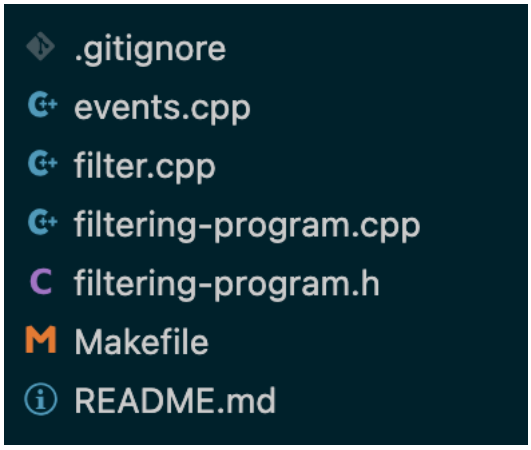


Taken screenshot of the output in the console

11.2 Appendix A.2 – Program Source Code

Figure 10

File structure of the benchmark program



The file structure of the program is as follows, with a separate file for the main program; filtering-program, one file for the filtering method; filter.cpp, and another for the method which generates the event array/list and the according checksum array/list. Then there is also the .h file which is a header file which holds all declarations, like the variables used across all files, or methods. Finally the Makefile, which contains instructions which tell the computer how to compile the code.

Figure 11

Contents of the header file

```cpp
// This file contains all shared declarations for a simulated event filtering program (scaled down version of what PrecisionBeam works on)

#pragma once // ensures that this file is included (loaded) only once in a single compilation, ensuring that the data-types wont be redefined causing errors

//including the necessary libraries
#include <cstddef> //gives the std::size_t type, which is an unsigned (positive and 0) integer type used for sizes and counts
#include <cstdint> //gives the std::uint16_t and std::uint32_t types, which are fixed-width unsigned integer types
#include <vector>

constexpr unsigned NUM_THREADS = 12;   // number of threads to be used for multithread experiments

// setting fixed unchangeable variables, holding general specifications, which will be used throughout the program
// constexpr means they cannot be changed while the program is running, and it also guarantees the compiler knows this value when compiling

constexpr std::size_t NUM_EVENTS = 30000;
// each event is set to 20,000 readings = 40 kB per event
constexpr std::size_t EVENT_READINGS = 20000;
// a vaild event that should be kept will get 12k strong readings (thus its kept when compared against the 10k threshold)
constexpr std::size_t SIGNAL_STRONG = 12000;
// an invalid event that will be discarded will get 2k strong readings (thus it is discared when compared against the 10k threshold)
constexpr std::size_t BACKGROUND_STRONG = 2000;
// readings below this are noise
constexpr uint16_t THRESHOLD = 1000;
// events with fewer strong readings are discarded
constexpr std::size_t MIN_STRONG = 10000;

// function declaration (done for the compiler to be informed of the existence of these functions and that the structs will be used there)
uint32_t sum_checksum(const std::vector<uint16_t>& event);
void generate_events(
    std::vector<std::vector<uint16_t>>& readings,
    std::vector<uint32_t>& checksum);
bool filter_event(std::vector<uint16_t>& event, uint32_t stored_checksum);
```

The header file contains the constant variables like the number of events, number of readings per event etc. all constexpr variables meaning that they are fixed when the program is compiled and cannot change while it runs, thus allowing for settings of the experiments to be kept in one place and changed when needed. Variables are stored as std::size_t which is the standard unsigned type (0<) for sizes and positions in arrays/lists; as uint16_t which is the smallest type that holds a 12-bit detector value, thus the threshold for comparing readings, is the same type, and checksums as uint32_t, since their sum can reach 81.9 million (12-bit means the largest possible value is 4,095, and the experimented value of readings is 20k, so 20,000 x 4,095 = 81,900,000). At the bottom, there are also three functions declared, sum_checksum, which adds up an event's readings to compute a checksum; generate_events, which creates the events and their checksums; and filter_event, which filters a single event and returns whether it is kept. Declaring them here, in the header, permits every source file that includes it to call them, while their code is written in separate source files, thus the filtering or event generation method can be called from the main program file, if the program includes the header, even though they're in different files.

Figure 12

Contents of the main file

```cpp
#include "filtering-program.h"
#include <chrono> // allows to count time
#include <functional> // allows for passing the original variable to a thread instead of its copy std:ref
#include <iostream> // allows for printing to the terminal std:cout
#include <thread> // allows for creating and running threads std::thread



// function which calls the filterring method for every event, designed for multiple threads, since it takes in only a portion of the events, based on first and last arguments
void filter_range(std::vector<std::vector<uint16_t>>& readings, std::vector<uint32_t>& checksum, std::size_t first, std::size_t last, std::size_t& kept) {
    kept = 0;

    // filtering function applied to every event
    for (std::size_t e = first; e < last; e = e + 1) {
        if (filter_event(readings[e], checksum[e])) {
            kept = kept + 1;
        }
    }
}



// the main method
int main() {

    //-------------------------------------------------------------------------------
    // GENERATING EVENTS
    // nested array which will store all events and readings, readings[i] would show an array containing all readings inside an event, readings[i][i] will show a particular reading of an event
    // uint16_t since each reading is maximum 12-bits
    std::vector<std::vector<uint16_t>> readings;
    // array which stores the checksum for each event, so checksum[i] stores the checksum for event i (so corresponds to readings[i])
    // uinit32_t because checksum adds all readings together, so 20k times 4,095 gives 81.9 million which is the highest possible, so not enough for 16 but enough for 32 bit which gives up to 4.29 billion
    std::vector<uint32_t> checksum;

    // readings = [
    //     [ 412, 2731, 87, 3390, 655, ... ],    // event 0    (signal: 12,000 readings >= 1000)
    //     [ 30, 655, 1904, 12, 498, ... ],      // event 1    (background: 2,000 readings >= 1000)
    //     [ 799, 2048, 4011, 233, 61, ... ],    // event 2    (signal)
    //     ...
    //     [ 215, 88, 3076, 540, 9, ... ]        // event 5999
    // ]
    //
    // checksum = [ 34120577, 13048211, 33987402, ..., 12961530 ]
    //              // event 0   event 1    event 2         event 5999



    // print
    std::cout << "Generating " << NUM_EVENTS << " events...\n";

    // record start time
    auto gen_starttime = std::chrono::steady_clock::now();

    // function to generate events and fill the two lists above
    generate_events(readings, checksum);

```

```
56      // record end time
57      auto gen_endtime = std::chrono::steady_clock::now();
58
59      // calculate the duration from start time and end time
60      std::chrono::duration<double> DurationGeneration = gen_endtime - gen_starttime;
61
62      // print
63      std::cout << "Generation took " << DurationGeneration.count() << " s\n\n";
64
65      // make an identical copy of the events and checksum list so each test filters its own fresh events
66      std::vector<std::vector<uint16_t>> readings_copy = readings;
67      std::vector<uint32_t> checksum_copy = checksum;
68
69      // the size of all th event data that goes through in MB used for printing later
70      double megabytes = NUM_EVENTS * EVENT_READINGS * 2 / 1000000.0;
71
72      //-------------------------------------------------------------------------------------------
73      // FILTERING EVENTS
74
75
76          //============================ 0 Baseline: filter every event in a plain loop on one core
            ==========================
77
78          std::size_t kept_0 = 0; // variable for printing
79
80          // record start time
81          auto starttime_0 = std::chrono::steady_clock::now();
82
83          //filtering function applied to every event
84          for (std::size_t e = 0; e < NUM_EVENTS; e = e + 1) {
85              if (filter_event(readings[e], checksum[e])) {
86                  kept_0 = kept_0 + 1;
87              }
88          }
89
90          // record end time
91          auto endtime_0 = std::chrono::steady_clock::now();
92
93          // calculate the duration from start time to end time
94          std::chrono::duration<double> DurationFiltering_0 = endtime_0 - starttime_0;
95
96
97          //========== 1 Multithreaded: split filtering events across threads which run simultaneously on multiple
            cores =============
98
99          // starts a list which will hold the running threads
100         std::vector<std::thread> threads;
101         // a list which stores the number of kept events for each thread
102         std::vector<std::size_t> kept_per_thread(NUM_THREADS);
103
104         // record start time
105         auto starttime_1 = std::chrono::steady_clock::now();
106
107         // loop runs once per thread, with every cycle a thread is created
108         for (unsigned t = 0; t < NUM_THREADS; ++t) {
109             // calculates the position of the first event
110             std::size_t first = t * NUM_EVENTS / NUM_THREADS;
111             // calculates the position of the last event, the last event for one thread will overlap with the first
                event for another thread, but the filter_range takes care of that
112             std::size_t last = (t + 1) * NUM_EVENTS / NUM_THREADS;
```

```
113
114             // push_back adds a new member to the threads list and resizes it as well so it can dynamically add
                elements to a list
115             // std:thread creates a thread with the first argument being the method each of the threads will run
116             // std:ref means it will take the original instead of making a copy
117             threads.push_back(std::thread(filter_range, std::ref(readings_copy), std::ref(checksum_copy), first,
                last, std::ref(kept_per_thread[t])));
118         }
119
120         // the method waits from executing the rest of the code until every thread has finished, it actually waits
            for the first thread, and by that time the rest would probably be done
121         for (unsigned t = 0; t < NUM_THREADS; ++t) {
122             threads[t].join();
123         }
124
125         // record end time
126         auto endtime_1 = std::chrono::steady_clock::now();
127
128         // calculate the duration from start and end time
129         std::chrono::duration<double> DurationFiltering_1 = endtime_1 - starttime_1;
130
131         // initialize a variable to store the number of events that were kept
132         std::size_t kept_1 = 0;
133         // traverse the kept_per_thread list and add every element and save under the kept variable
134         for (unsigned t = 0; t < NUM_THREADS; t = t + 1) {
135             kept_1 = kept_1 + kept_per_thread[t];
136         }
137
138
139         //============================================== Performance Calculations
            ==============================================
140
141         // counting speedup between single and multi threaded experiment
142         double speedup = DurationFiltering_0.count() / DurationFiltering_1.count();
143
144         // Amdahl: speedup = 1 / (f + (1 - f) / N), solved for f gives f = (N / speedup - 1) / (N - 1)
145         // calculating the serial fraction for multithreaded experiment using Amdahl's Law
146         double serial_fraction = (static_cast<double>(NUM_THREADS) / speedup - 1.0) / (NUM_THREADS - 1.0);
147         // parallel proportion is the remaining part of the work that benefits from more threads
148         double parallel_proportion = 1.0 - serial_fraction;
149
150         // predicting the speedup on 28 cores from the measured serial fraction using Amdahl's Law
151         constexpr unsigned PREDICTED_CORES = 28;
152         double predicted_speedup = 1.0 / (serial_fraction + parallel_proportion / PREDICTED_CORES);
153
154         //=================================================== Printing
            ==============================================
155
156
157         // print for single threaded 0
158         std::cout << "Experiment 0: baseline (one core, no threads)\n";
159         std::cout << "Kept " << kept_0 << " of " << NUM_EVENTS << " events (" << 100.0 * kept_0 / NUM_EVENTS << "%)
            \n";
160         std::cout << "Time: " << DurationFiltering_0.count() << " s (" << megabytes / DurationFiltering_0.count() <<
            " MB/s)\n";
161
```

```
162         // print for multithreaded 1
163         std::cout << "Experiment 1: scaling with " << NUM_THREADS << " threads\n";
164         std::cout << "Kept " << kept_1 << " of " << NUM_EVENTS << " events (" << 100.0 * kept_1 / NUM_EVENTS << "%)
            \n";
165         std::cout << "Time: " << DurationFiltering_1.count() << " s (" << megabytes / DurationFiltering_1.count() <<
            " MB/s)\n";
166
167         // print for calculations
168         std::cout << "Speedup: " << speedup << "x\n";
169         std::cout << "Serial fraction (Amdahl): " << serial_fraction * 100 << "%\n";
170         std::cout << "Parallel proportion (Amdahl): " << parallel_proportion * 100 << "%\n";
171         std::cout << "Predicted speedup on " << PREDICTED_CORES << " cores (Amdahl): " << predicted_speedup << "x\n";
172
173     return 0;
174 }
```

The main file contains all library imports at the top, which ensure the necessary functions are provided, such as the header file or the function to create threads. Then the first method is a filtering method designed for the multi-threading experiment, described later. Then there is the main method, which stores the central code which connects all methods. It starts by creating two arrays/lists, one for storing the generated events, and one for storing the generated checksum for each event, lines 28-44 explain their structure. Then there is a print and on line 51 is the method which measures the time with steady_clock, which is a clock that only moves forward, thus suited to

measuring durations. Followed by which the generate_events method is called (see figure 13), where the two earlier created arrays are passed as arguments. That method fills the two arrays with generated variables and then on line 57 the end time is measured, followed by another variable which holds the duration, by subtracting start time from end time. Then the print with the time it took to generate the events, followed by copying the two generated arrays into new variables, thus there are two sets of data the program can run on. Then there is another variable which calculates the total size of the generated event array, based on the values set in the header file. Followed by that is the block of code responsible for performing the first experiment by running the filtering method on the array single thread, in lines 76-95. First is the variable kept_0, which is a variable set to 0 that stores the number of events the filter keeps which is printed at the end to verify the expected result is produces, thus the function works correctly. The same time measuring is applied here, where in between of the time measuring methods there is a loop that goes through every event, in the event array (readings), and for every event it runs the filter_event method from the filter.cpp file, parsing in the array of readings, which is represented by readings[e], since readings is a nested array, which holds reading for every event, and its corresponding checksum, which is one integer. The way this method works will be explored in figure 14, but the method modifies the array under the position of that event and returns either true or false, thus its inside an if statement, which if true, increases the kept_0 variable. The same time end is computed and duration calculated and saved under another variable, which will be printed at the end, together with the kept_0.

Then there is the second block, which represents the second experiment, where multiple threads are run. It begins by creating two lists, first one holding the running threads, and the kept_per_thread, which is sized to the number of threads set in the header file, which will store how many events were kept by each thread, which each thread will write to its own position, thus, no two threads write to the same position. Then the start time is recorded in the same way, under a different variable, followed by a for loop, which loops once per thread, with each cycle having two lines as seen and described in the comments (lines 109-112), which calculate the first and last event for that thread, based on the number of events and number of threads stated in the header file. So for example for 4 threads and 100 events, it will iterate 4 times, giving the first thread, first of 0, and last of 25, the second thread will have 25 and 50, the third will have 50 and 75, and the last will have 75 and 100 (the filtering function itself ensures the last and first aren't processed by 2 threads, as can be seen later). Then we have the function which adds a new element to the earlier created threads list using push_back, inside which we create a thread using the std::thread, where we specify the first argument to filter_range, which is the method the trhead runs, followed by the list of events and checksum, which are copies of what was used in experiment 0, then the first and last computed earlier and the position in the list corresponding to the thread, which will store how many events that thread kept. The events, checksums and counter are passed with std::ref, so the thread works on the originals instead of making its own copy, making the program generally more efficient. From here we can imagine we go into the filter_range menthod, which is at the top of the file (lines 9-19), which contains the same flirting block as experiment 0, but instead of looping through every event and performing the filter_event method on every single one, it only loops from the first and last, sent as arguments, which depend on the thread we are running. Because < last is used instead of <= the last event won't be considered, thus the filtering is done one 0-24, 25-49, 50-74, 75-99, which is correct, as when we specify there to be 100 events it will create a list of indexes from 0-99, as it starts from 0, thus first event is at position 0, and there would be no event under position 100. Then if the filter_event method decides to keep the event, it outputs true, and the kept counter is incremented, which is actually mapped to the argument passed in at the last position, which is the one position of kept_per_thread list. As the loop to create threads ends when we reach the last thread, the threads are now running and the main code is simultaneously executed, thus the second loop is reached, where another for loop which has as many loops as there is threads, which calls the .join() on every thread, which is a method that makes the main program wait until the first thread is done executing, and its iterated on each thread number, thus it waits until all threads are finished. Then the end time is recorded and the duration is computed, just like in the previous experiment, and saved under a new variable. Then another loop adds all elements of the kept_per_thread, into one kept_1 variable, which will also be printed later.

Then there is a block which performs result calculations. The speedup is calculated by dividing the single-threaded duration by the multi-threaded duration, using .count() which returns each duration in seconds as a decimal number. Then the serial fraction is calculated by reordering Amdahl's law using the speedup and the number of threads (which represent cores) which need a static_cast<double> to turn the number of threads into a decimal and get the exact result rather than having it rounding to a whole numbers, as originally NUM_THREADS was an integer. The parallel portion is calculated from the serial portion, and the predicted speedup uses a new variable

which suggest how many cores we want the predicted speedup for, stored under PREDICTED_CORES set to 28, through using the same Amdahl's Law equation.

Finally, there are the statements which print all values to the terminal regarding the two experiments 157-174 using cout, and then the main method is closed.

Figure 13

Contents of the events file

```
// Generates the synthetic detector events
// Every event has the same number of readings. Signal events (15%) contain
// enough strong readings to pass the filter; background events (85%) do not.
// 1% of events get a wrong checksum so the corruption check has work to do (only background events get corrupted)
#include "filtering-program.h"

#include <algorithm>
#include <random>

// method which adds up all readings of one event (used to create and later check against checksum)
// unit32_t is the variable type it gives out, the same as each variable inside 'checksum' array
// const means it will only read the variable parsed in as the argument, and whats after suggests the type
// followed by & means that the variable isn't copied, which improves performance and event means it will save whats parsed in under a variable called "event"
uint32_t sum_checksum(const std::vector<uint16_t>& event) {
    uint32_t sum = 0;

    // loop which iterated through every reading for the event passed in above saved as "event"
    for (uint16_t reading : event) {
        sum += reading;
    }
    return sum;
}

// method which generates events
// void means the function wont output anything thus only change the variables that were parsed in from the man file which calls this function
// in brackets the two arguments will be passed from the main function, under which the generated data will be saved
void generate_events(
    std::vector<std::vector<uint16_t>>& readings,
    std::vector<uint32_t>& checksum
) {
    readings.resize(NUM_EVENTS);
    checksum.resize(NUM_EVENTS);

    //initializing a random number generator called rng, 12345 inside brackets (seed) suggests it will produce the same sequence of numbers every run
    std::mt19937 rng(12345); // seed: the same events every run

    // nested loop for populating the nested array of events and checksum
    for (std::size_t e = 0; e < NUM_EVENTS; ++e) {
        readings[e].resize(EVENT_READINGS);

        // 1 identify if the reading is supposed to be valid or not

        // this variable will hold the number of strong readings
        std::size_t strong_count;

        // 15 in 100 events are set to be valid, thus if the event number divided by 100 has a remainder of less than 15 it is set to have a number of strong reading which will allow it to be a kept event
```

```
47          if (e % 100 < 15) {
48              strong_count = SIGNAL_STRONG;      // signal event: 12,000 strong readings
49          } else {
50              strong_count = BACKGROUND_STRONG;  // background event: 2,000 strong readings
51          }
52
53          // 2 fill the readings list for the event
54
55          // loop which traverses each reading of this event, and saves a generated value under
            it
56          for (std::size_t i = 0; i < EVENT_READINGS; ++i) {
57              // the first readings are always the 'strong' ones, which would be kept if
                compared against the threshold
58              // if the current reading is bigger than the number of strong ones set earlier,
                this event is set to a noisy reading
59              if (i < strong_count) {
60                  readings[e][i] = THRESHOLD + rng() % 3096; // adds threshold with a randomly
                    generated number between 0-3096 (where the maximum possible is equal to 4096
                    due to the 12-bit readings)
61              } else {
62                  readings[e][i] = rng() % (THRESHOLD - 100); // generates a number between
                    0-900 to ensure this number is below the threshold and can be discarded during
                    filtering
63              }
64          }
65
66          // 3 mix readings, to have random positions
67
68          // loop to traverse every reading going from the back
69          for (std::size_t i = EVENT_READINGS - 1; i > 0; --i) { //number of events is minused 1
            as in an array with 6000 events, there will be 5999 indexes, as it starts from 0
70
71              // initializing a number which will store a randomly generated value between 0 and
                i +1 since the last number is not included in generation so without +1 the max
                number would be 5998
72              std::size_t j = rng() % (i + 1);
73
74              // creating a temporary variable for a reading, which will store the current
                reading
75              uint16_t temp = readings[e][i];
76              // swapping the reading under i with the reading under j
77              readings[e][i] = readings[e][j];
78              readings[e][j] = temp;
79          }
80
81          // 4 checksum is made wrong on exactly 1 of every 100 events, of a background
            (discarded) event thus 15% kept event output is maintained
82
83          // checksum list is populated with the sum_checksum function which produces a correct
            checksum for every event
84          checksum[e] = sum_checksum(readings[e]);
85          // for every event at position ..99 the checksum is corrupted, we are corrupting only
            invalid events, as the valid events are stored at positions ....0-..15
86          if (e % 100 == 99) {
87              checksum[e] = checksum[e] + 1;
88          }
89
90      }
91  }
```

At the beginning the library imports are present, including the header file and two libraries which provide functionalities, like a random number generator.

The events file contains two methods, sum_checksum, which is used to calculate the checksum of the parsed in event. It takes 1 argument, the array of readings of one event, and returns a 32-bit variable, which is the checksum calculated by adding every reading together. It iterates through the passed in array, and adds all values to a sum, a 32-bit variable initialized at the beginning, which after the loop ends, is returned.

Followed by this is the generate_event method which doesn't return anything, but modifies the two variables which it takes in as arguments, where the first one is the nested array/list of events and the second one is the array of checksums, which are passed in from the main function as in figure 12, which passes two empty structures. The

method first resizes both accordingly to the set number of events, then initializes a random number generator with mt19937, called rng, with a fixed seed of 12345, so the same events are generated in every run and the results of different trials can be compared.

Then a loop starts which iterates through every position of readings, which represents every event. First, current position, representing and event, which holds a list of readings, is resized accordingly with the number of readings set in the header file. Then a variable which will hold the number of strong readings is set, thus this variable is sensitive to every event, and an if statement is used to assign the number of strong readings which for the first 15 events out of 100, thus the program will only work with number of events that are multiples of 100, are set to be valid, thus will have a number of strong readings that will allow them to pass the threshold test lates, and for every other one it is set to a number below that threshold. Both of these variables are set in the header. Then the next loop fills the array of readings with numbers, every event will have some number of weak readings, thus the strong_count is used to identify if the current reading should be randomized between 1000 and 4096 or under 900. This randomization is done with the use of the rng (randomizer), where for the weak readings it just generates a number between 0-900 and for the strong readings it generates a number between 0-3096, and then adds it to the THRESHOLD variable set in the header file.

Then there is another loop, until this point, we have a readings[e] list under which the first events readings[e][1, 2, 3, ...] are strong readings, and the second portion is weak (the proportion weak to strong is dependent on if the event is in the first 15%). Thus, the events are mixed, therefore unpredictable in this next loop. This is done by iterating through every reading again, from last to first, where a random number is generated with the rng, and a temporary variable to hold the reading being swapped is then made of the same type as the readings. Followed by this the current reading is saved into this temporary variable and the reading under the index of this randomly generated variable is set at the place of the current variable. This is done for every reading, thus shuffling all readings.

Lastly the checksum is computed, by using the sum_checksum method which adds all readings together as mentioned previously, so every event gets a checksum in the checksum array, which correspond to each other through the indexes. Then an if statement, sets every 99th event to be corrupted by adding 1 to that event's checksum. That way we ensure that this corrupted event is not a valid one, and the 15% share of events to be kept remains correct.

Figure 14

Contents of the filter file

```cpp
#include "filtering-program.h"

bool filter_event(
    std::vector<uint16_t>& event,
    uint32_t stored_checksum
) // both arguments are not read only
{
    //invalid events return false

    // discards events with corrupted checksum, through calculating each event checksum again and comparing it
    against the stored checksum, which is parsed from main function, where its generated alongside each event
    if (sum_checksum(event) != stored_checksum) {
        return false;
    }

    // check each reading against threshold and sum up the number of total valid readings under "strong"
    std::size_t strong = 0;

    // traverse each event's readings and count how many valid readings there are, and clear out the
    for (std::size_t i = 0; i < event.size(); i = i + 1) {
        if (event[i] >= THRESHOLD) {
            event[strong] = event[i]; // saves the valid reading under the next valid position
            strong = strong + 1; //increments the number of valid readings
        }
    }
    event.resize(strong); // cuts the list to be the length of strong readings

    // checks if the event is valid based on the number of strong readings and the set minimum, if not it discards it
    if (strong < MIN_STRONG) {
        return false;
    }
    return true;
}
```

At the beginning the header file is included, and then there is the filter_event method which returns a bool, boolean so true or false, and takes in two arguments where the first one is the array of readings of one event, and the second one is the checksum, as seen in figure 12. The filtering function starts by checking if the event is corrupted, by using sum_checksum as in figure 13 passing in the array as the argument, adds all readings together, and then outputs a 32-bit value, which is compared against the set checksum, and if they don't match the method returns false immediately, discarding the event.

Then there is a variable which stores the number of valid readings, starting at 0, and then a loop iterates through every reading comparing it against the threshold, if it passes it is saved in the same readings array under position given by strong, which is then increased by one. Thus, the strong readings are moved to the front of the array in their original order, and the weak ones are overwritten, and after the last reading the loop ends, and the array containing readings "event" is resized to be the size of the number of strong events, which removes the leftover readings at the end.

Finally, the if statement checks if this event has enough strong readings to be considered as valid, if not the method returns false, otherwise it returns true.